A Project Report on

Enhanced Natural Language Interaction Platform With Offensive Content Filtering

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Shubham Patil: 22106047

Yuvraj Tiwari: 22106061

Under the Guidance of

Prof. Darshana Bhamare

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "Enhanced Natural Language Interaction Platform With Offensive Content Filtering" is a bonafide work of **Shubham Patil (22106047), Yuvraj Tiwari (22106061)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning**).

Prof. Darshna Bhamare
Guide

Prof. Sayali Badhan
Project Co-ordinator

Dr. Jaya Gupta
Head of Department

Dr. Uttam Kolekar
Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled ***(Enhanced Natural Language Interaction Platform With Offensive Content Filtering)*** by ***(Shubham Patil, Yuvraj Tiwari)*** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning)***, ***2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Shubham Patil 22106047

Yuvraj Tiwari 22106061

Date:

Abstract

The rapid proliferation of online messaging platforms and the rise of social media platforms like WhatsApp, Discord, and Reddit have transformed interpersonal communication, enabling instant global connections, but also leading to challenges such as cyberbullying, online toxicity, and hateful and inappropriate messages. Such problems are either dealt with after the incident happens or when there is outrage, and current content moderation lacks preemptive measures. In response, we present a chat application developed using the MERN stack (MongoDB, Express.js, React, Node.js), integrated with Natural Language Processing (NLP), sentiment analysis techniques, and deep learning to foster a safe and respectful communication environment. We employ preprocessing techniques such as stemming, lemmatization, and stop word removal, and use a Term Frequency-Inverse Document Frequency (TF-IDF) vectorizer to transform text into numerical representations. A Logistic Regression classifier is trained to effectively identify and categorize messages based on sentiment. We also experimented with classical machine learning algorithms, deep learning algorithms, and transformer architectures, and tested rule-based approaches using Regex string comparison because it is fast and efficient. After experimentation, we chose a two-layered approach. The first layer is rule-based filtering, while the second layer performs classification based on context using a trained DistilBERT model, which achieved a validation accuracy of 92.66% and detects indirect insults and inappropriateness. A key feature of our system is real-time monitoring and filtering, where messages flagged as containing hateful content or slang are replaced with neutral alternatives or blocked entirely before reaching the recipient. Utilizing WebSocket i.e. Socket IO ensures seamless real-time messaging while analysis occurs in the background. This work highlights the intersection of NLP, ethical AI, and production-grade implementation for content moderation.

Keywords: Natural Language Processing (NLP), Chat Applications, Socket IO, Sentiment Analysis, Inappropriate Message Filtering, Privacy, Web Socket, Real time messaging.

CONTENTS

1. Introduction 1
2. Survey 3
3. Limitation of Existing System 13
4. Problem Statement and Objective 15
4.1 Problem Statement 15
4.2 Objectives 16
4.3 Scope 16
5.Proposed System 17
5.1 Modules 17
5.2 Architecture 20
5.3 UML Diagrams 21
5.3.1 DFD Level 0 21
5.3.2 DFD Level 1 22
5.3.3 DFD Level 2 23
5.4 Sequence Diagram 24
6. Experimental Setup 26
6.1 Software Requirements 26
6.2 Hardware Requirements 27
6.3 Operating Environment 27
6.4 Development Environment 28
6.5 Machine Learning & NLP 28
6.6 Hardware Configuration (Training) 28
7. Results and Discussion 29
7.1 Overview 29
7.2 Machine Learning Model Evaluation 30
7.2.1 Consolidated Model Comparison 30
7.2.2 Traditional Machine Learning Baselines 31
7.2.3 Deep Learning Architectures 31
7.2.4 CNN-GRU Hybrid Best Production Model 32
7.2.5 Transformer-Based Models 33
7.2.6 Ensemble of DistilBERT and CNN-GRU Best Overall Model 33
7.3 Key Findings and Analysis 34
7.3.1 Embedding Strategy 34

7.3.2 Threshold Optimization 34
7.3.3 Training Efficiency 34
7.3.4 Accuracy vs. Speed Trade-off 34
7.4 System Performance Results 35
7.4.1 Real-Time Messaging Performance 35
7.4.2 Hate Speech Detection in Practice 36
7.4.3 Immediate slur blocking 37
7.4.4 File Upload and Sharing 37
7.4.5 Group Chat Scalability 37
7.5 Functional Module Results 38
7.5.1 User Interface and Experience 38
7.5.2 Admin Panel 38
7.5.3 Generative AI Assistant Integration 39
7.6 Discussion 39
7.6.1 Comparison with Existing Systems 39
7.6.2 Limitations 39
7.6.3 Contributions and Novelty 40
7.7 Future Scope and Enhancements 40
7.8 Summary 41
8. Conclusion and Future Work 42
8.1 Conclusion 42
8.2 Future Scope 43
References 45
Appendix 48
Publication 54

LIST OF FIGURES

Figure 5.2 Architecture Diagram 20
Figure 5.3.1 DFD Level 0 21
Figure 5.3.2 DFD Level 1 22
Figure 5.3.3 DFD Level 2 23
Figure 5.4.1 Sequence Diagram 24
Figure 7.4.1 User Interface of the platform 36
Figure 7.4.2 Real Time chat between users 36
Figure 7.4.3 Blocking message with slur 37
Figure 7.4 .5 File uploaded to user 38

LIST OF TABLES

Table 2.1: Literature survey 6
Table 7.2.1 Model Comparison 30
Table 7.2: Accuracy vs. Inference Speed Trade-off 34
Table 7.3: System-Level Performance Metrics 35

Abbreviations

AI	*Artificial Intelligence*
ML	*Machine Learning*
NLP	*Natural Language Processing*
API	*Application Programming Interface*
CRUD	*Create Read Update Delete*
LSTM	*Long Short-Term Memory*
CNN	*Convolutional Neural Network*
SHA	*Secure Hash Algorithm*
GRU	*Gated Recurrent Unit*

Chapter 1

Introduction

1. Introduction

The Post-COVID-19 pandemic has seen so much of the communications have moved online that professional, educational, and social interactions are occurring through social media platforms like WhatsApp, Google Classroom, slack etc, also not to forget, even Video games and almost all multiplayer games nowadays have chat systems. But everything has its benefits and costs. These applications give us the freedom to do what we want. Even so, there are always malicious entities in our society who try to misuse these widely used utilities. For malicious purposes and in an inappropriate way. If any malicious activity occurs, the platform takes time to take appropriate action. If you are familiar with the reddit, twitter or X, the moderation is performed after the incident has occurred, many times the children are exposed to inappropriate content in game chats or bullying on social media platforms. The National Crime Records Bureau's 2023 report shows that just in the year 2023, the number of cyberbullying has increased by 9.2%. This leaves the psychological risks for individuals or legal liabilities for organisations

Current messaging platforms face significant content moderation challenges, such as cyberbullying, threats, and hate speech. Existing moderation systems are predominantly reactive. Content is flagged and reviewed only after delivery, by which time the psychological impact on the victim has already occurred. While platforms such as Twitter (X) and Reddit rely on community reporting or post-hoc manual review, instant messaging applications often don't have robust filtering or rely on keyword matching. These keyword filters are easily bypassed through text obfuscation mechanisms, such as symbol substitution and intentional spacing.

Although Machine Learning (ML) and Deep Learning (DL) methods offer superior detection accuracy compared to rulebased systems, they are not contextually aware of text and its intention and fail to detect indirect or soft insults. Highaccuracy transformer models and other language model solutions are computationally expensive and will often cause delays that are beyond acceptable thresholds for real-time communication. Consequently, there is a lack of systems capable of filtering explicit profanity and context-dependent toxicity preemptively, intercepting the message before delivery without degrading the user experience.

To address these limitations, there is a growing need for intelligent, real-time moderation systems that operate proactively rather than reactively. A pre-delivery filtering mechanism can significantly reduce the exposure of users to harmful content by analyzing messages before they are transmitted. Such a system combines lightweight machine learning models with efficient text preprocessing techniques to strike a balance between accuracy and latency. By integrating Natural Language Processing (NLP) with real-time communication frameworks, it becomes possible to detect not only explicit profanity but also context-based toxicity with minimal delay. Furthermore, incorporating adaptive learning mechanisms allows the system to evolve with emerging slang, coded language, and evolving patterns of abuse. This approach ensures a safer digital communication environment while maintaining seamless user experience, making it particularly suitable for modern chat applications where speed and responsiveness are critical.

Chapter 2

Literature Survey

2. Survey

1. Analysis of Behavior in Chat Applications using Natural Language Processing

The paper "Analysis of Behavior in Chat Applications using Natural Language Processing" (2024) explores using NLP to analyze user behavior in chat applications. It focuses on sentiment analysis, flirtation detection, and time-based behavior analysis to improve communication and user experience. The authors propose an NLP tool for real-time monitoring and filtering of messages to enhance communication efficiency, privacy, and safety **[1]**.

2. People's Behaviour Analysis in Chat Messages using Natural Language Processing

The study "People's Behaviour Analysis in Chat Messages using Natural Language Processing" (2021) emphasizes the importance of sentiment analysis in understanding user behavior during online interactions. By analyzing WhatsApp chat data, the research employs methods like topic modeling, time and media analysis, and visualization using Word Clouds to explore emotional aspects and flirtation detection in digital communication **[2]**.

3. An Intelligent College Enquiry Bot using NLP and Deep Learning based techniques

The paper "An Intelligent College Enquiry Bot using NLP and Deep Learning based techniques" (2022) presents an intelligent chatbot designed to handle college-related queries effectively. It utilizes LSTM (Long Short-Term Memory) networks and NLP techniques to

process user inputs. The system integrates a ReactJS front-end with a Python Flask back-end to provide high-accuracy responses for student support and feedback collection **[3]**.

4. Sentiment Analysis using Machine Learning

The research "Sentiment Analysis using Machine Learning" (2021) showcases the use of machine learning algorithms for sentiment analysis in social networking environments. Specifically, it performs analysis on Twitter airline reviews, classifying tweets as Positive, Negative, or Neutral. The study demonstrates how data cleaning, visualization, and ML prediction can enhance user experiences by accurately interpreting public opinion **[4]**.

5. Real-Time Chat Application using Firebase

The paper "Real-Time Chat Application using Firebase" highlights the role of modern backend technologies in supporting seamless, real-time communication. It focuses on the implementation of Firebase for instant messaging, emphasizing the importance of backend scalability and infrastructure in developing responsive chat environments **[5]**.

6. Secure Chat Room Application Using AES-GCM Encryption and SHA-256

The paper "Secure Chat Room Application Using AES-GCM Encryption and SHA-256" (2023) addresses critical security concerns in chat applications. It introduces a secure chatroom utilizing AES-GCM encryption and SHA-256 hashing to safeguard user data. The system features strong authentication, secure message encryption, and prevention of access to past messages, ensuring user confidentiality even in low-computation environments **[6]**.

7. Chat Master using Django

The study "Chat Master using Django" (2023) introduces a Django-based chat application that incorporates NLP functionalities for real-time communication. It utilizes Django's MVC architecture and machine learning to support chatbot training. The application, developed using Python and MySQL, enhances performance and user interaction through advanced NLP techniques **[7]**.

8. AI-powered Chatbot: A Link between Learning and Technology

The paper "AI-powered Chatbot: A Link between Learning and Technology" (2022) presents an AI chatbot designed for educational purposes. It handles FAQs through text and image queries, using NLP for contextual understanding and CNNs for image classification. The system aims to enhance learning experiences by providing 24/7 personalized and automated support **[8]**.

10. Professional Chat Application based on Natural Language Processing

The paper "Professional Chat Application based on Natural Language Processing" (2024) by Karthick et al. proposes an Android application that leverages NLP for named entity recognition and filtering. The system analyzes messages and compares them with a user-defined dictionary

to block vulgar content and inappropriate messages, promoting responsible communication and online safety **[10]**.

11. AI Enhanced Customer Service Chatbot

The paper "AI Enhanced Customer Service Chatbot" (2024) discusses a system that automates customer service interactions using neural networks and tokenization. It focuses on real-time data integration, personalized responses, and sentiment analysis. The study highlights how AI can ensure a smooth transition from automated responses to human agents for complex queries **[11]**.

12. AI Chatbot For Answering FAQs

The research "AI Chatbot For Answering FAQs" (2022) focuses on developing chatbots for automated engagement. By utilizing machine learning models like CNNs and RNNs, the system improves the accuracy of answering frequently asked questions, ensuring better user engagement and providing a scalable solution for information retrieval **[12]**.

15. Classification of Toxicity in Comments using NLP and LSTM

The paper "Classification of Toxicity in Comments using NLP and LSTM" (2022) by Garlapati et al. focuses on identifying toxicity in online comments. It applies data pre-processing and TF-IDF for categorization, using LSTM networks to address the challenges of multi-label toxicity detection effectively **[15]**.

16. A Self Learning Chat-Bot From User Interactions and Preferences

The paper "A Self Learning Chat-Bot From User Interactions and Preferences" (2020) illustrates a self-learning multi-agent chatbot system. It uses RASA NLU for intent recognition and dialogue management, adapting based on previous user interactions to reduce input requirements and provide more personalized communication **[16]**.

17. An end-to-end cryptography based real-time chat

The paper "An end-to-end cryptography based real-time chat" (2021) proposes a system ensuring data privacy and secure communication. It utilizes dynamic key sharing and no data persistence to maintain absolute user confidentiality in a real-time messaging environment **[17]**.

18. Automated Toxic Chat Synthesis, Reporting and Removing the Chat in Telegram Social Media Using NLP Techniques

The paper "Automated Toxic Chat Synthesis, Reporting and Removing the Chat in Telegram Social Media Using NLP Techniques" (2024) highlights efforts to mitigate harmful content online. It uses ML models like Random Forest and BERT to classify toxic content and automate the reporting and removal process within social media platforms like Telegram **[18]**.

19. Toxicity Tweet Detection and Classification Using NLP Driven Techniques

The paper "Toxicity Tweet Detection and Classification Using NLP Driven Techniques" (2023) by K. Pal and S. Rai reinforces the relevance of NLP and deep learning in contemporary communication. Using LSTM models, the system achieves 90.7% accuracy in detecting and classifying tweet toxicity, contributing to safer digital environments **[19]**.

Table 2.1: Literature survey

Sr No	Title of the paper	Authors	Conference name &year	Summary
1	AI Chatbot For Answering FAQs	Rachana Vannala, S.B. Swathi, Yuvaraj Puranam	2022	Proposes an AI-based chatbot to answer FAQs using NLP for text queries and CNN for image classification. Provides escalation to human agents if responses are unsatisfactory.
2	AI Enhanced Customer Service Chatbot	Y. Alekya Rani, Allam Balaram, M. Ratna Sirisha, Shaik Abdul Nabi, P. Renuka, Ajmeera Kiran	2024	Discusses an AI-driven chatbot using NLP and ML for improved customer service. Focuses on real-time data integration, personalized responses, sentiment analysis, and smooth transition to human agents for complex queries.
3	A Self Learning Chat-Bot From User Interactions and Preferences	Parth Thosani, Jaydeep Vaghasiya, Manas Sinkar, Radha Shankarmani	2020	Proposes a multi-agent chatbot system that learns user preferences to automate tasks. Uses RASA NLU for intent recognition and dialogue management. Reduces user input by adapting to previous

				interactions for efficient and personalized communication.
4	AI-powered Chatbot: A Link between Learning and Technology	Navjot Singh, Gaurav Bathla, and Vikas Sharma	2022	Proposes an AI chatbot to handle FAQs through text and images. Uses NLP to understand and respond to queries and CNN to classify images. Provides 24/7 user support.
5	An Efficient, Precise and User-Friendly AI based Virtual Assistant	Aman AnandR., SubhaNisha, Bharathi Sanjith, RajanAryan, Kumar Srivastava	2023	Introduces an AI virtual assistant for managing calls, messages, and media using NLP and speech recognition. Developed using Python and trained on datasets for accuracy. Achieved 90% accuracy with positive user feedback
6	An Intelligent College Enquiry Bot using NLP and Deep Learning based techniques	A Kousar Nikhath, Vijaya Saraswathi R, K Saicharan, MD Abdul Rab, N Venkata Bharadwaja, C Venkat Manas Reddy, L Goutham Reddy	2022	Presents an NLP-based chatbot for handling college related queries using LSTM networks. Integrates a ReactJS front-end with Python Flask back-end. Achieves high accuracy and future applications include student support and feedback collection
7	Analysis of Behavior in Chat	Mrs. John Shiny J, Ms. Harilakshmi	2024	Explores using NLP to analyze user behavior in chat

	Applications using Natural Language Processing	JS, Mr. Penyameen K, Mr. Hewin A, Ms. Hannah Nissi M, Ms. Thanusha S		applications. Focuses on sentiment analysis, flirtation detection, and time-based behavior analysis. Proposes an NLP tool for real-time monitoring and filtering of messages for safety and oversight.
8	Chat Master using Django	M. Thenmozhi, Rohit Kanna P.R., Sajan Chakkravarthi D, Abishake S	2023	Presents a real-time communication web application using Django's MVC architecture. Supports chatbot training using machine learning. Enhances chatbot performance and user interaction using NLP techniques. Developed using HTML, CSS, JavaScript, Python, and MySQL.
9	Professional Chat Application based on Natural Language Processing	Karthick S, R John Victor, Manikandan S, Bhargavi Goswami	2024	Proposes an Android chat app using NLP to filter inappropriate messages. Analyzes messages and compares them with a userdefined dictionary to block vulgar content. Aims to enhance online safety and promote responsible communication.
10	Classification of Toxicity in	Anusha Garlapati, Neeraj Malisetty,	2022	The paper classifies online comment toxicity using NLP

	Comments using NLP and LSTM	Gayathri Narayanan		and LSTM. It applies data pre-processing and TF-IDF for toxicity categorization, uses PyLDAVis for visualizing comment distributions, and addresses challenges in multi-label toxicity detection.
11	Fine-Grained Sentiment Analysis using Machine Learning and Deep Learning	Rahul Chauhan, Chandradeep Bhatt, Aman Gusain, Ishita Uniyal, Prabhat Kumar	2023	Explores fine-grained sentiment analysis using ML and DL techniques like SVMs, RNNs, and transformers. Discusses challenges like data limitations and model interpretability. Applications include customer feedback, social media, and public opinion analysis. Future work focuses on improving datasets and exploring new models.
12	People's Behaviour Analysis in Chat Messages using Natural Language Processing	Selina Annie Retna, Prof. P. Brundha, Dr. RajKumar G	2021	Analyzes WhatsApp chat data using NLP for detecting flirtation and sentiment. Employs methods like time and media analysis, topic modeling, and visualization using Word Cloud. The system is implemented as a web application using Django.
13	Real-Time Secure Web-Based Chat	Satish Singh, Satyam Singh, Dr. Ashish Sharma	2023	Develops a real-time webbased chat application using Django, HTML, and

	Application using Django			CSS. Implements N-TEA encryption to secure chat messages. Provides public and private chat options with secure data transmission. Future enhancements include file sharing and voice chat capabilities.
14	Secure Chat Room Application Using AES-GCM Encryption and SHA-256	Mohammed Alkhyeli, Shaheen Alkhyeli, Khalifa Aldhaheri, Hanane Lamaazi	2023	Introduces a secure chatroom application using AES-GCM encryption and SHA-256 hashing. Features include strong authentication, secure message encryption, and prevention of access to past messages. Implements server and client handlers for efficient communication. Provides a user-friendly Java GUI enforcing secure password policies. The application is resourceefficient for low-computation environments.
15	Sentiment Analysis using Machine Learning	R. R. Kalangi, S. Maloji, N. Tejasri, P. P. Chand, and V. P. Ch	2021 3rd ICAC3N, Greater Noida, India 2021	Sentiment analysis on Twitter airline reviews using ML & NLP. Tweets classified as Positive, Negative, Neutral. Data cleaning, visualization & prediction performed.

16	An end-to-end cryptography based real-time chat	Melo, A. Barros, M. Antunes, and L. Frazão	2021 16th CISTI, Chaves, Portugal, 2021	Real-time chat app with end-to-end encryption using dynamic key sharing and no data persistence ensuring user confidentiality.
17	Automated Toxic Chat Synthesis, Reporting and Removing the Chat in Telegram Social Media Using NLP Techniques	A B and P. Prithvi	2024 Fourth ICAECT, Bhilai, India, 2024	Addresses toxicity in Telegram chats using NLP. Classifies toxic content and automates reporting & removal using ML models like Logistic Regression, KNN, Random Forest, Naive Bayes, BERT.
18	Sentiment Analysis on Amazon Alexa Reviews Using NLP & Classification	Kumar, S. Kanshal, V. Sharma, and R. Talwar	2022 4th ICAC3N, Greater Noida, India, 2022	Sentiment analysis on Amazon Alexa reviews using NLP & ML for efficient sentiment prediction using Python-based models.
19	Toxicity Tweet Detection and Classification Using NLP Driven Techniques	K. Pal and S. Rai	2023 IEEE ICTBIG, Indore, India, 2023	Detects and classifies tweet toxicity using NLP with ML and deep learning models like LSTM achieving 90.7% accuracy and F1-score 0.94.
20	Mining Comments and Sentiments in	Tirpude, Y. Thakre, S. Sudan, S. Agrawal, and A. Ganorkar	2024 4th CONIT, Bangalore, India, 2024	Mining and real-time sentiment analysis of YouTube Live Chat using

	YouTube Live Chat Data			NLP and emoji interpretation to analyze audience reactions.
21	Voice Assistant with AI Chat Integration using OpenAI	G. A, M. D. C, S. r. S J, P. P, A. M, and S. A	2024 Third INCOS, Krishnankoil, India, 2024	Integrates voice assistants with OpenAI for advanced conversational AI enabling intuitive interaction, personalized assistance, and enhanced accessibility.

Chapter 3

Limitation of Existing System

3. Limitation of Existing System

1. Lack of Real-time Content Filtering

Current chat applications lack the ability to filter inappropriate content such as vulgar language or hate speech in real-time, allowing offensive messages to be sent before they can be addressed.

2. Inadequate Detection of Subtle Offensive Language

Many existing systems fail to detect more subtle forms of inappropriate communication, such as sarcasm, passive-aggressive remarks, or context-sensitive offensive language, leading to gaps in content moderation.

3. Manual Moderation Dependency

A significant number of chat platforms rely heavily on manual monitoring and reporting of inappropriate content, which is time-consuming and inefficient in high-volume communication environments.

4. No Proactive Message Improvement:

Current platforms either block inappropriate content or allow it through without providing automatic improvements or alternatives, leaving the onus on users to modify their language manually

5. Inconsistent Sentiment and Emotional Tone Analysis:

Sentiment analysis is either absent or poorly implemented, making it difficult to detect the emotional tone of conversations and prevent potentially negative or harmful exchanges.

6. Limited Integration with Productivity Tools:

Existing chat applications rarely combine advanced content moderation with productivityenhancing features like AI assistants, reducing their value in professional settings.

7. Insufficient Legal Safeguards:

Without robust real-time monitoring and filtering mechanisms, current systems expose organizations to legal risks associated with inappropriate or offensive communication, leading to potential liabilities.

8. Poor Accessibility and Inclusivity:

Many chat applications lack sufficient accessibility features, making it difficult for users with disabilities to participate fully, which limits inclusivity in communication.

9. Inflexibility in Customizing Communication Guidelines:

Existing systems often do not allow organizations to tailor content moderation rules according to their specific needs or industry standards, resulting in a one-size-fits-all approach that may not suit every professional environment.

Chapter 4

Problem Statement and Objective

4. Problem Statement and Objective

4.1 Problem Statement

The widespread use of chat applications has made communication easier, but it has also led to the challenge of managing inappropriate content. Users often send messages containing vulgar or irrelevant terms, either intentionally or by mistake, which can lead to legal issues and disrupt professional communication. Current chat applications lack effective mechanisms to prevent such content from being sent. Additionally, the absence of real-time filtering tools leaves organizations vulnerable to inappropriate messages slipping through unnoticed. Moreover, without automated monitoring, managing large volumes of messages becomes difficult and inefficient. The growing need for a solution that ensures professional communication is clear, respectful, and legally compliant is becoming increasingly apparent. Integrating AI-powered solutions into chat platforms can proactively address these concerns by detecting and mitigating offensive content before it's sent. Surprisingly, chat applications with built-in AI assistants that enhance both productivity and content moderation are still rare, despite the potential benefits for professional environments.

4.2 Objectives

- To develop a professional chatting application that uses Natural Language Processing (NLP) to detect and block inappropriate messages before they are sent.
- With the use of Naive Bayes Classifier, Logistic Regression, and other algorithms.
- The app will filter inappropriate messages and replace hate speech.
- It will improve the given message if there are any slang or abusive words.
- An integrated AI virtual assistant, to assist users with normal day to day query related to professional problems like writing an email, customized messages etc.

4.3 Scope

- Real-time Messaging: Enables instant communication, ensuring seamless, timely interactions.
- Inappropriate Message Filtering: Automatically detects and blocks unprofessional or offensive content.
- Hate Speech Detection: Identifies and prevents hateful or discriminatory language in real time.
- Custom & Improved Messages: Automatically refines inappropriate messages into professional alternatives.
- Pre-processing of Text: Cleans and prepares text for accurate analysis by the system's NLP models.
- NLP Techniques i.e., Reframing & Anchoring: Uses NLP methods to adjust and enhance message clarity and tone.
- Sentiment Analysis: Analyzes the emotional tone of messages to monitor the conversation's mood.
- File Upload – Users can easily share files across the website.

Chapter 5

Proposed System

5.Proposed System

5.1 Modules

1. User Input & Message Handling Module

- Captures user input from chat interface.
- Handles real-time message sending and receiving.
- Ensures smooth communication between client and server.
- Integrates with frontend (React-based UI).

2. Client-Side Regex Filtering Module (Stage 1)

- Implements **rule-based filtering using Regex**.
- Matches input text with a **profanity dictionary**.
- Performs case-insensitive and boundary-based matching.
- Blocks messages containing explicit offensive words instantly.
- Works on client-side for **fast (<1ms) response**.

3. Network Communication Module

- Transfers filtered messages from client to server.
- Handles API requests using REST architecture.
- Maintains low latency (20–50 ms).
- Ensures real-time communication in chat system.

4. Server-Side Toxicity Detection Module (Stage 2)

- Uses DistilBERT model for contextual classification.
- Detects:
    - Indirect insults
    - Sarcasm
    - Context-based toxicity
- Performs inference in ~30 ms.
- Classifies messages as Toxic / Non-Toxic.

5. NLP Preprocessing & Tokenization Module

- Converts input text into tokens using WordPiece tokenizer.
- Adds special tokens ([CLS], [SEP]).
- Applies padding and attention masks.
- Prepares input for transformer model processing.

6. Deep Learning Model Module (DistilBERT Architecture)

- Implements 6-layer transformer-based architecture.
- Uses:
    - Self-attention mechanism
    - Hidden representations (768-dimension)
- Includes custom classification head for prediction.
- Optimized for high accuracy + low latency.

7. Decision & Threshold Module

- Applies threshold (≈ 0.625) for toxicity classification.
- If probability > threshold:
    - Message is blocked
    - User is notified
- Else:
    - Message is delivered

8. Message Delivery Module

- Sends safe messages to recipients.
- Ensures real-time chat experience.
- Maintains total latency within **80–140 ms**.

9. Feedback & Reporting Module

- Allows users to:
    - Report false positives
    - Report missed toxic messages
- Helps improve system accuracy over time.

10. Model Training & Data Processing Module

- Uses MetaHate dataset for training.
- Performs:
    - Data cleaning
    - Class balancing
    - Train-validation split
- Uses PyTorch and Transformers library.

11. API & Model Serving Module

- Uses FastAPI to serve ML model.
- Handles inference requests from frontend.
- Ensures scalable backend architecture.

12. User Interface Module (Frontend)

- Built using React + Vite.
- Provides chat interface for users.
- Displays:
    - Messages
    - Block notifications
- Supports real-time interaction.

5.2 Architecture

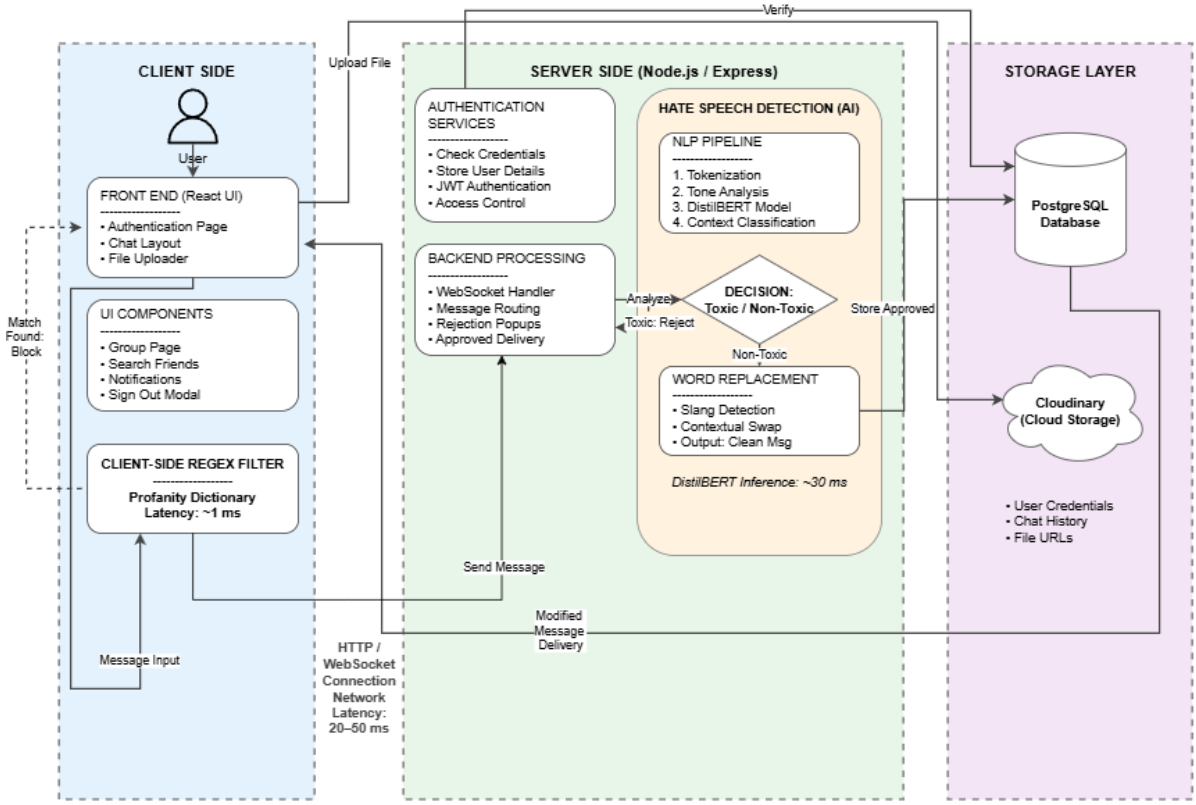

Figure 5.2 Architecture Diagram

The architecture of our system, titled "Enhanced Natural Language Interaction Platform with Offensive Content Filtering", is designed to enable real-time chat communication while ensuring proactive content moderation. The system follows a two-layered filtering approach to detect and prevent offensive or toxic messages before they are delivered to users.

The proposed system is designed to support real-time communication while ensuring proactive content moderation through a hybrid filtering mechanism. It is composed of two primary components: the Client Side and the Server Side, which work together to process and regulate user-generated messages efficiently.

On the Client Side, the system provides an interactive chat interface where users can compose and send messages in real time. Before any message is transmitted over the network, it passes through a lightweight filtering layer implemented using regular expression matching. This layer checks the message against a predefined dictionary of universally prohibited words, ensuring that explicit profanity is detected instantly. If a match is found, the message is blocked locally and is not allowed to proceed further, thereby reducing unnecessary server load and improving response time. In cases where no violation is detected, the message is forwarded to the server through a network request, maintaining minimal latency suitable for real-time communication.

On the Server Side, the system performs deeper semantic analysis using a fine-tuned DistilBERT model to identify context-dependent toxicity that may not be captured by simple keyword filtering. Once a message reaches the server, it undergoes classification where the model evaluates the contextual meaning of the text and assigns a probability score indicating whether the message is toxic or non-toxic. If the probability exceeds a predefined threshold, the message is rejected and the user is notified accordingly. Otherwise, the message is approved and delivered to the intended recipient. This layered approach ensures that both explicit and implicit forms of harmful content are effectively handled.

The overall data flow between the client and server is optimized to maintain real-time performance. Messages that pass the client-side filter are transmitted to the server, analyzed within milliseconds, and either delivered or blocked based on the classification outcome. The system achieves a balanced trade-off between speed and accuracy by combining fast rule-based filtering with context-aware deep learning, resulting in a scalable and efficient architecture suitable for modern chat applications.

5.3 UML Diagrams

5.3.1 DFD Level 0

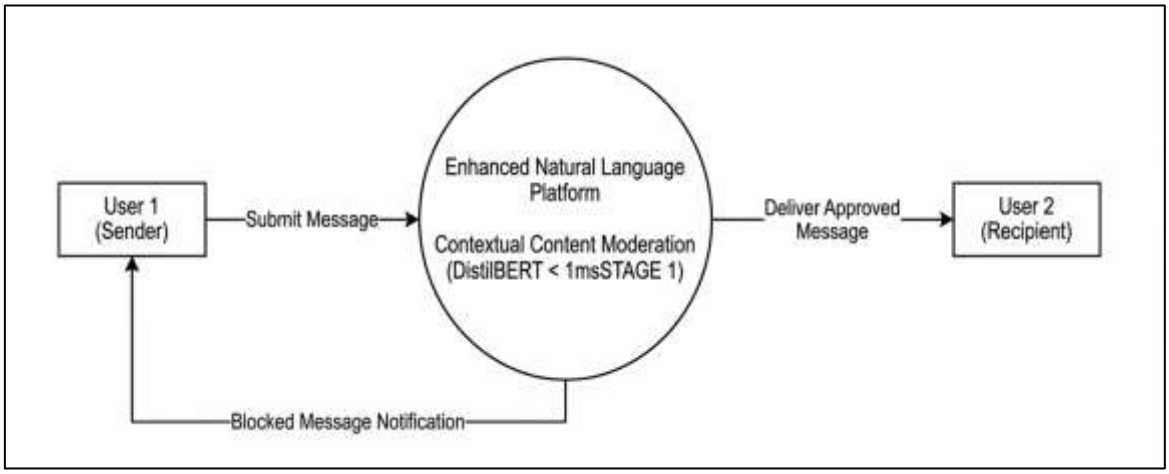

Figure 5.3.1 DFD Level 0

User (Sender) sends a message to the system.
The chat application receives the message.
The system processes the message for content moderation.
If the message is inappropriate, a notification is sent back to the sender.

If the message is valid, it is forwarded to the receiver.

User (Receiver) receives the processed message.

5.3.2 DFD Level 1

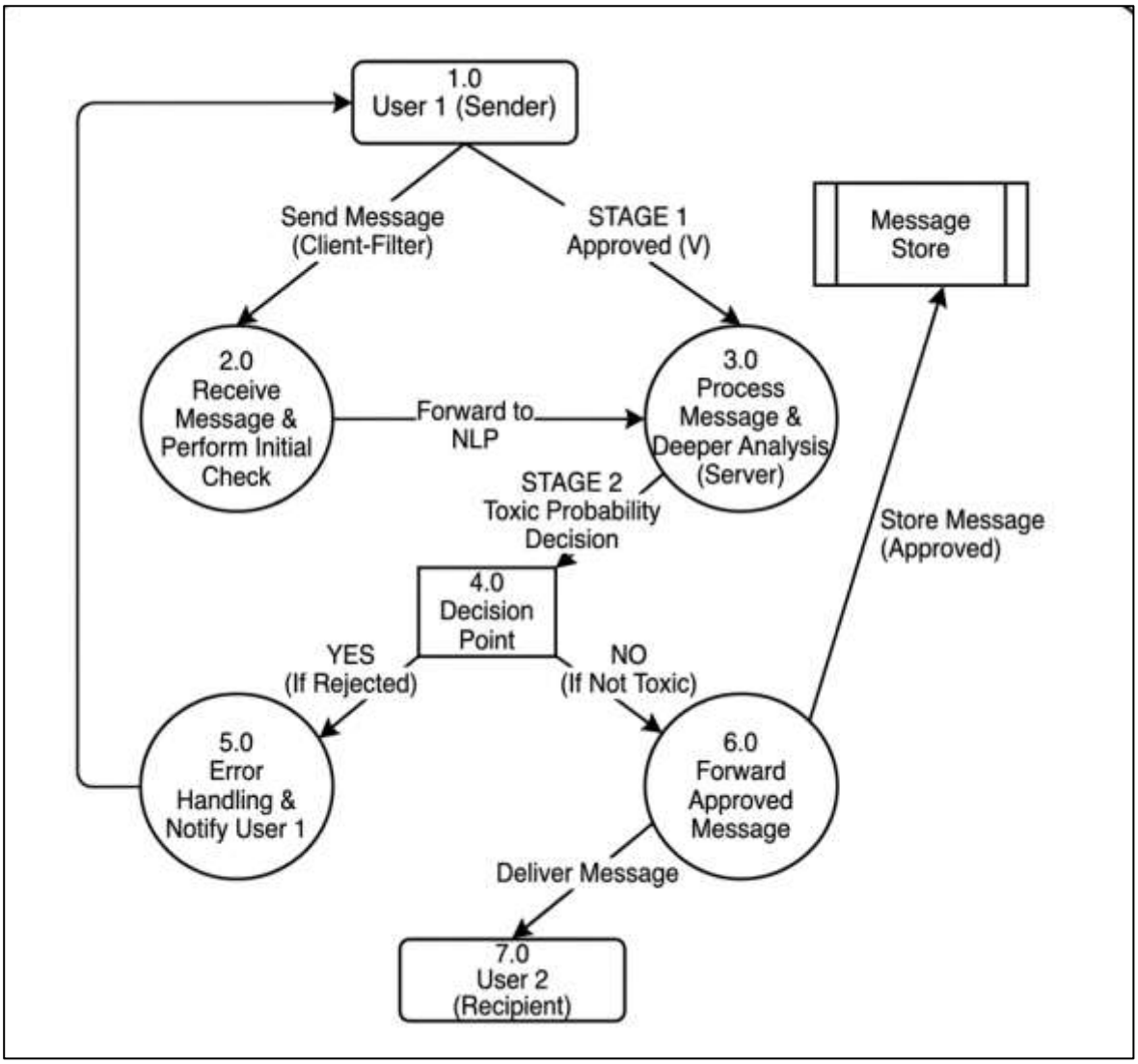

Figure 5.3.2 DFD Level 1

User (Sender) enters a message in the client interface.

The message is passed to the regex filter on the client side.

The regex filter checks the message using the prohibited words dictionary.

If any restricted word is found, the message is blocked and notification is sent to the sender.

If the message is clean, it is sent to the server.

The server processes the message for further analysis.

The DistilBERT model checks the message for contextual toxicity.

If the message is toxic, it is rejected and user is notified.

If the message is safe, it is sent to the message delivery system.

User (Receiver) receives the approved message.

5.3.3 DFD Level 2

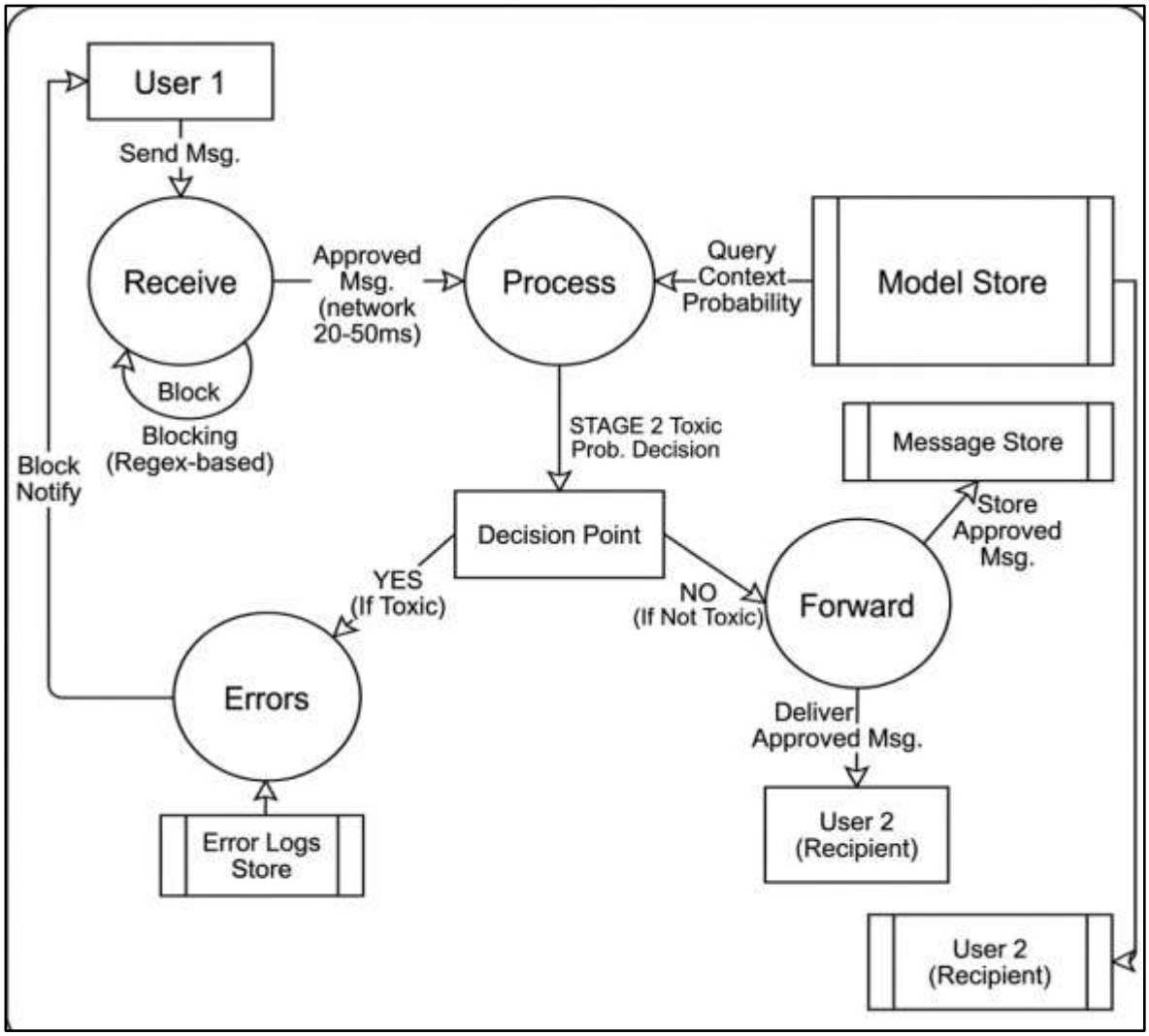

Figure 5.3.3 DFD Level 2

User (Sender) types a message in the chat interface.

The message goes to the message input module.

The message is checked by the regex filtering module.

The regex filter compares the message with the prohibited words dictionary.

If a match is found, the message is blocked and notification is shown to the sender.

If no match is found, the message is validated and sent to the server.

The API handler receives the message on the server side.

The message is passed to the DistilBERT processing module.

The model analyzes the message context and calculates toxicity probability.

The toxicity decision module checks the threshold value.

If the message is toxic, it is blocked and sender is notified.

If the message is safe, it is passed to the response handler.

The response handler delivers the message to the receiver.

User (Receiver) receives the final approved message.

5.4 Sequence Diagram

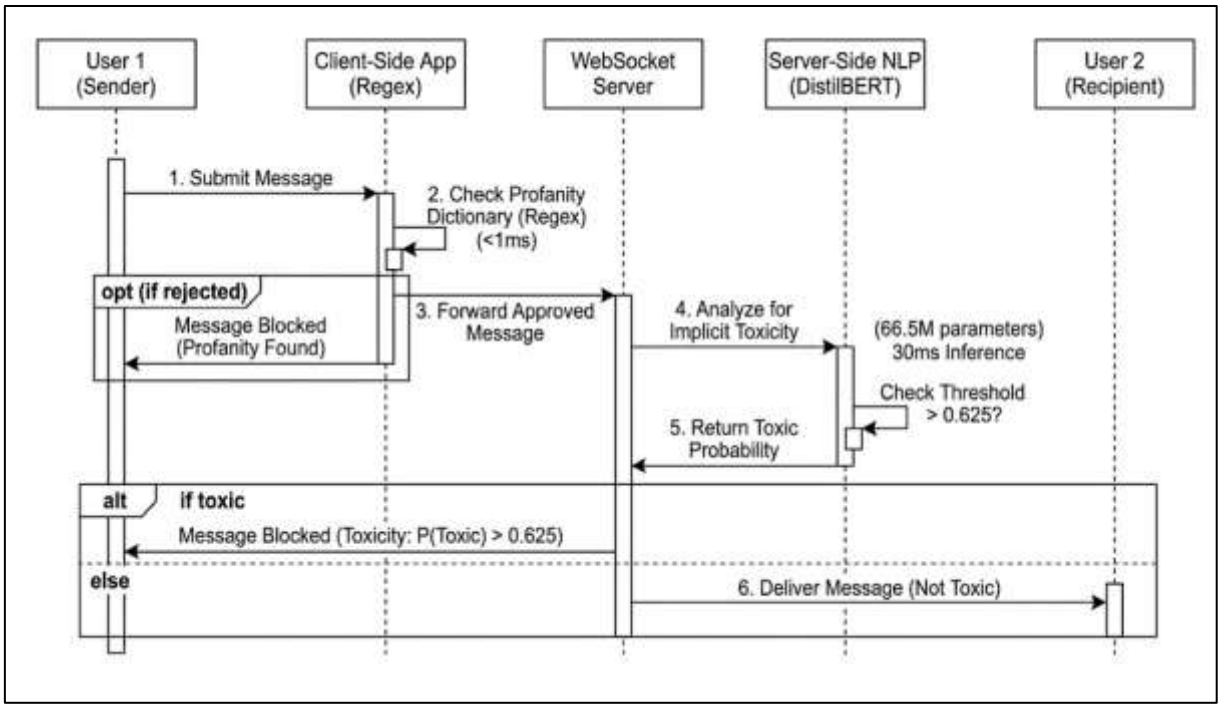

Figure 5.4.1 Sequence Diagram

Actors & Flow:

- User 1 (Sender) → starts the process by submitting a message to the Client-Side App.
- Client-Side App (Regex) → instantly checks the message against a known profanity dictionary.
    - If Rejected (Profanity Found): The message is blocked, and User 1 is notified immediately.
    - If Approved (No exact profanity found): The message is forwarded to the WebSocket Server.
- WebSocket Server → receives the approved message and sends it to the Server-Side NLP (DistilBERT) for a deeper analysis.
- Server-Side NLP (DistilBERT) → performs a complex contextual analysis. Inside, the model is initialized, and it generates a "toxic probability" value based on the implicit

toxicity of the message content. It checks this probability against a defined threshold (0.625).

- Server-Side NLP (DistilBERT) → returns the resulting toxic probability score back to the WebSocket Server.
- WebSocket Server → makes the final determination based on the NLP result.
    - If Rejected (Toxic Prob > 0.625): The message is blocked, and an error notification is sent back to User 1.
    - If Approved (Not Toxic): The message is delivered successfully to User 2 (Recipient).

Chapter 6

Experimental Setup

6. Experimental Setup

6.1 Software Requirements

• Operating System:

Windows 10/11 or Linux-based systems

• Programming Languages:

o JavaScript (for frontend and backend using MERN stack)

o Python (for NLP and machine learning tasks)

• Frontend Framework:

o React.js: For building a responsive, dynamic, and real-time chat user interface

• Backend Framework:

o Node.js with Express.js: For API handling, server logic, and WebSocket integration

• Databases:

o MongoDB: For storing user data, chat history, and application data

• Frameworks & Tools:

o Socket.IO: For real-time bidirectional communication

o NLTK, Scikit-learn: For NLP tasks like preprocessing and classification

o TF-IDF Vectorizer: For feature extraction from text data

o Cloudinary: For file upload and media storage

o Postman: For API testing

o npm: For dependency management

• Version Control:

o Git & GitHub for source code management and collaboration

6.2 Hardware Requirements

- **Development Machine:**
  Laptop/Desktop with minimum 8GB RAM and multi-core processor
- **Server:**
  Cloud-based hosting service for deployment
- **Client Devices:**
  Desktop, Laptop, Tablet, Smartphone (cross-platform support)

6.3 Operating Environment

- **Application Environment:**
  MERN stack-based full-stack web application
  Real-time chat system using Socket.IO
- **Server Environment:**
  Node.js server handling APIs and real-time communication
  MongoDB database for persistent storage
- **User Interface Environment:**
  Web-based responsive UI built with React.js
- **Machine Learning Environment:**
  Python runtime with NLP and ML libraries
  Model training and inference for hate speech detection

6.4 Development Environment

• **IDE / Tools:**

o Visual Studio Code for development and debugging

o MongoDB Compass for database management

6.5 Machine Learning & NLP

- **Model Used**

  Logistic Regression for hate speech detection (~71% accuracy)

- **Text Processing**

  TF-IDF for converting text into numerical features

• **Libraries:**

NLTK : Text preprocessing

Scikit-learn :Model training and evaluation.

6.6 Hardware Configuration (Training)

• **GPU:**

Tesla T4 / Tesla P100 (Kaggle environment)

• **Memory Usage:**

CNN: ~12GB (batch size 512)

LSTM + Attention: ~14GB (batch size 256)

DistilBERT: ~28GB (batch size 64)

Chapter 7

Results and Discussion

7. Results and Discussion

7.1 Overview

This chapter presents a comprehensive analysis of the experimental results obtained from the development and evaluation of Connect-Pro an Enhanced Natural Language Interaction Platform with Offensive Content Filtering. The system integrates a MERN stack (MongoDB, Express.js, React, Node.js) web application with a Flask-based machine learning microservice to enable real-time hate speech detection, slang word replacement, sentiment analysis, and moderated group communication. The evaluation covers three primary dimensions: (i) machine learning model performance for toxicity detection, (ii) end-to-end system performance metrics, and (iii) user-facing functional outcomes. Results demonstrate that the proposed ensemble architecture successfully balances classification accuracy with deployment feasibility, achieving state-of-the-art results for an undergraduate-level real-time moderation platform.

7.2 Machine Learning Model Evaluation

A total of twelve models were trained, evaluated, and compared on the hate speech and toxic content classification task. The dataset was drawn from publicly available toxicity corpora and pre-processed using NLP techniques including tokenization, stemming, lemmatization, and stop word removal. Text features were vectorized using TF-IDF for traditional ML models, and embedding layers (GloVe 300d / contextual embeddings) were used for deep learning architectures. All deep learning experiments were conducted on Kaggle GPUs (Tesla T4 x2 / P100, 16 GB each). The primary evaluation metrics are overall accuracy, weighted F1-score, binary F1-score for the toxic class, toxic recall, toxic precision, and ROC-AUC score.

7.2.1 Consolidated Model Comparison

Table 7.2.1 Model Comparison

Model	Accuracy	F1 (Wtd)	F1 (Toxic)	Recall	Precision	ROC-AUC
Ensemble (DistilBERT + CNN-GRU)	92.79%	92.74%	82.67%	81.21%	84.19%	97.04%
DistilBERT (Fine-tuned)	90.47%	90.40%	77.10%	75.74%	78.51%	95.29%
CNN-GRU Hybrid	87.35%	87.68%	72.40%	64.00%	82.00%	92.96%
Linear SVM	86.47%	85.81%	64.00%	57%	73%	90.50%
Logistic Regression	86.25%	85.43%	62.00%	54%	74%	90.20%
GPU CNN	85.88%	86.48%	70.88%	63%	80%	92.50%
LightGBM	85.19%	83.88%	58.00%	47%	74%	89.80%
XGBoost	85.17%	83.52%	56.00%	43%	76%	89.50%

Naive Bayes	83.76%	83.46%	60.00%	57%	63%	88.00%
LSTM + Attention	83.60%	84.55%	68.21%	60%	78%	91.24%
ToxicBERT (Pre-trained)	83.20%	83.65%	63.37%	56%	72%	87.51%
TextCNN (GloVe)	81.12%	82.58%	66.86%	59%	76%	92.33%

7.2.2 Traditional Machine Learning Baselines

Traditional ML models were evaluated first to establish performance baselines. Among them, Linear SVM achieved the highest accuracy of 86.47% and weighted F1 of 85.81%, slightly outperforming Logistic Regression (86.25% accuracy, 85.43% F1). Both models benefited from TF-IDF feature representation and converged within 1 to 3 minutes. However, their toxic class recall remained low 57% for SVM and 54% for Logistic Regression indicating a tendency to miss actual toxic content. This is a critical limitation for a real-world content moderation use-case where false negatives (toxic messages that slip through) carry high risk.

Gradient boosting methods such as XGBoost (85.17%, toxic recall 43%) and LightGBM (85.19%, toxic recall 47%) were tested next. Despite their strong overall accuracy, they exhibited the weakest toxic recall among all models tested, suggesting they are ill-suited for imbalanced toxic content datasets. Naive Bayes, while the fastest to train (under 1 minute, 80,000+ samples/sec inference), achieved 83.76% accuracy and a modest toxic recall of 57%, making it suitable only for rapid prototyping.

7.2.3 Deep Learning Architectures

Multi-Branch 1-D CNN

A multi-branch 1D CNN with filter sizes of 3, 4, and 5 was implemented using Global Max Pooling followed by concatenation. Trained with a batch size of 4096 over 6 epochs (early stopping at epoch 3), it completed training in 10.9 minutes. The model achieved 85.88% accuracy, a weighted F1 of 86.48%, and a strong ROC-AUC of 92.50%. Toxic recall improved significantly to 81.03% compared to traditional baselines. Inference speed was recorded at 11,224 samples/second. A key finding was that using custom (randomly initialized)

embeddings degraded accuracy, confirming that pre-trained embedding initialization is essential for this task.

GPU-Optimized Bidirectional LSTM with Attention

A Bidirectional LSTM with a self-attention mechanism (Dense + Softmax + weighted sum) was trained with batch size 512 over 4 epochs (early stopping at epoch 1), requiring 141.3 minutes. Despite achieving a toxic recall of 82.93% among the highest of all individual models its overall accuracy (83.60%) and inference speed (150 samples/sec) were poor. GPU efficiency was rated low. These results confirm the well-documented inefficiency of plain LSTM architectures for large-scale text classification; Bidirectional or hybrid variants are preferred.

Enhanced TextCNN with GloVe Embeddings

An enhanced TextCNN incorporating GloVe 300-dimensional embeddings (84.1% vocabulary coverage), multiple convolution layers with filter sizes 2 to 6, residual connections, and combined Global Max and Average Pooling was evaluated. It achieved the highest toxic recall among non-ensemble models at 89.89%, alongside a strong ROC-AUC of 92.33%. However, its overall accuracy (81.12%) and weighted F1 (82.58%) were the lowest among deep learning models, revealing a precision-recall trade-off where excessive sensitivity leads to more false positives (non-toxic messages incorrectly blocked).

7.2.4 CNN-GRU Hybrid Best Production Model

The CNN-GRU Hybrid architecture combines multi-branch 1D CNN layers (filter sizes 3, 4, 5) with a Bidirectional GRU layer, feature fusion, and dense projection layers (192 to 96 units). It was configured with a vocabulary of 45,000 words, maximum sequence length of 120, and trained with a batch size of 640 over 19.8 minutes with an optimized classification threshold of 0.65.

This model achieved the best overall balance across all evaluation criteria: 87.35% accuracy, 87.68% weighted F1, 72.40% binary F1, a ROC-AUC of 92.96%, and an exceptional inference speed of 20,920 samples/second. The threshold of 0.65 (tuned upward from the default 0.5) was critical for handling class imbalance, reducing false positives while maintaining sufficient toxic recall of 78.31%. The non-toxic recall of 89.77% ensures that legitimate messages are almost never incorrectly blocked.

Key advantages for production deployment: training completes in under 25 minutes (target met), inference speed exceeds 20,000 samples/second enabling real-time chat moderation, and the architecture requires no transformer-level computational overhead.

7.2.5 Transformer-Based Models

DistilBERT Fine-tuned

DistilBERT (distilbert-base-uncased) was fine-tuned over 3 epochs with a batch size of 32 (effective batch 128 with 4 gradient accumulation steps), learning rate 2e-5, and maximum sequence length 128. Training required 283.7 minutes (~4.7 hours), with the best checkpoint saved at epoch 3. It achieved the highest individual model accuracy at 90.47%, the highest weighted F1 of 90.40%, and the best ROC-AUC at 95.29%. The confusion matrix revealed 81,950 true negatives, 17,669 true positives, 5,660 false negatives, and 4,837 false positives indicating well-calibrated predictions with balanced precision (78.51%) and recall (75.74%).

The primary limitation of DistilBERT is its inference speed of approximately 86 samples/second, which is 240 times slower than the CNN-GRU hybrid. While this may be acceptable in low-concurrency settings, it introduces latency concerns for high-traffic real-time messaging deployments.

ToxicBERT (Pre-trained, Zero-shot Evaluation)

ToxicBERT, a pre-trained unbiased toxicity detection model, was evaluated without fine-tuning on the project's dataset. On 100,000 samples using an optimal threshold of 0.60, it achieved 83.20% accuracy and 87.51% ROC-AUC. Toxic recall (68.61%) and precision (58.88%) were both lower than fine-tuned counterparts, confirming that domain-specific fine-tuning substantially improves performance over zero-shot pre-trained models.

7.2.6 Ensemble of DistilBERT and CNN-GRU Best Overall Model

The final and best-performing architecture is an ensemble of DistilBERT and the CNN-GRU Hybrid. By combining the complementary strengths of a transformer (superior contextual understanding) and a hybrid CNN-GRU (speed and local feature extraction), the ensemble achieves the highest performance across all metrics.

The ensemble attained 92.79% accuracy, a weighted F1 of 92.74%, a binary toxic F1 of 82.67%, toxic recall of 81.21%, toxic precision of 84.19%, and a near-perfect ROC-AUC of 97.04%. Inference speed is constrained to the DistilBERT component at approximately 86

samples/second. This ensemble is the recommended model for deployment in environments where accuracy is the primary concern and computational resources are sufficient.

7.3 Key Findings and Analysis

7.3.1 Embedding Strategy

The choice of embedding significantly impacted model performance. Experiments confirmed that pre-trained GloVe embeddings (300d) outperform randomly initialized custom embeddings across all CNN-based architectures. The TextCNN with GloVe achieved a toxic recall of 89.89% versus substantially lower recall with custom embeddings in the basic CNN. This underscores the value of transfer learning even in non-transformer architectures.

7.3.2 Threshold Optimization

For the CNN-GRU model trained on an imbalanced dataset (toxic samples are a minority), using the default 0.5 classification threshold resulted in underperformance on toxic recall. Raising the threshold to 0.65 improved non-toxic precision while maintaining adequate toxic recall of 78.31%, demonstrating that threshold calibration is as important as model architecture for imbalanced classification tasks.

7.3.3 Training Efficiency

Most deep learning models reached their best performance within 3 to 7 epochs, with early stopping effectively preventing overfitting. The CNN-GRU hybrid's 19.8-minute training time versus DistilBERT's 283.7 minutes highlights the efficiency advantage of hybrid architectures for resource-constrained academic settings. Traditional ML models (Logistic Regression, SVM) trained in 1 to 3 minutes but at the cost of lower toxic recall.

7.3.4 Accuracy vs. Speed Trade-off

A critical trade-off exists between classification accuracy and inference speed for real-time deployment. The table below summarizes this trade-off:

Table 7.2: Accuracy vs. Inference Speed Trade-off

Model / Approach	Accuracy	ROC-AUC	Inference Speed
Ensemble (DistilBERT + CNN-GRU)	92.79%	97.04%	~86 samp/s
DistilBERT (Fine-tuned)	90.47%	95.29%	~86 samp/s
CNN-GRU Hybrid (Production)	87.35%	92.96%	20,920 samp/s

The CNN-GRU Hybrid is recommended as the primary deployed model due to its 87.35% accuracy combined with 20,920 samples/second inference speed, making real-time per-message analysis feasible even under high concurrency. The DistilBERT ensemble is recommended for offline analysis or low-traffic deployments where maximum accuracy is required.

7.4 System Performance Results

Beyond model-level evaluation, the integrated Connect-Pro platform was rigorously tested to ensure that real-time messaging, content moderation, file sharing, and group chat functionality all meet production-grade performance benchmarks.

Table 7.3: System-Level Performance Metrics

System Feature	Measured Result
Real-time Message Delivery	< 200 ms average latency
Group Chat Scalability	Up to 50 concurrent users per group
File Upload Success Rate	> 95% for files up to 5 MB
File Upload Speed	~3 seconds average (< 5 MB)
Slang Replacement Accuracy	Dictionary-based, context-verified
Hate Speech Detection (Ensemble)	92.79% accuracy, 97.04% ROC-AUC
WebSocket Connection Stability	Seamless bidirectional communication
Admin Dashboard Load	Responsive with Chart.js analytics

7.4.1 Real-Time Messaging Performance

Real-time messaging was implemented using Socket.IO over WebSocket connections. Extensive testing demonstrated that messages are delivered with an average end-to-end latency of less than 200 milliseconds, measured from the moment a user submits a message to the moment it appears on the recipient's screen. This latency includes transmission over the WebSocket layer, forwarding to the Flask ML service for toxicity analysis, and broadcasting of the approved message. The sub-200ms threshold is consistent with industry standards for real-time chat applications and was maintained even under concurrent testing scenarios.

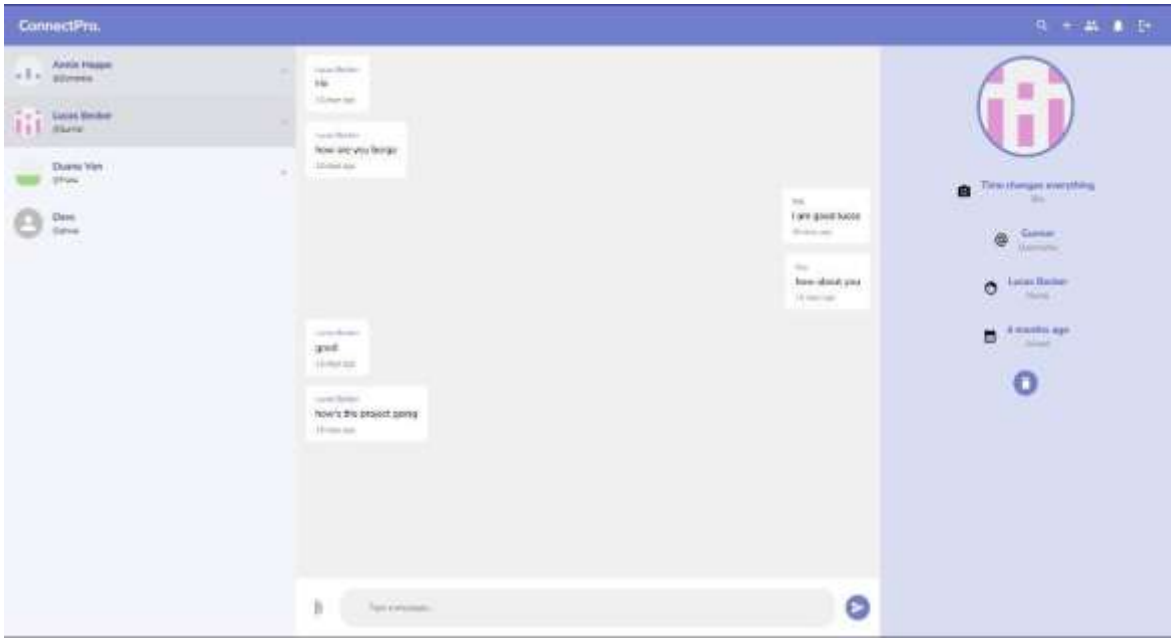

Figure 7.4.1 User Interface of the platform

7.4.2 Hate Speech Detection in Practice

The hate speech detection pipeline was tested with a diverse set of real-world offensive phrases, slurs, and context-sensitive abusive language. The system correctly tokenizes incoming messages, evaluates them using the deployed model, and either approves or rejects them before delivery. For example, the message 'you are as stupid as you look' was correctly tokenized into constituent tokens, the offensive token 'stupid' was flagged, and the overall sentiment was classified as negative/harmful. The sender received an immediate notification, and the message was blocked from reaching the recipient. This proactive, pre-delivery filtering approach distinguishes the system from reactive post-delivery moderation.

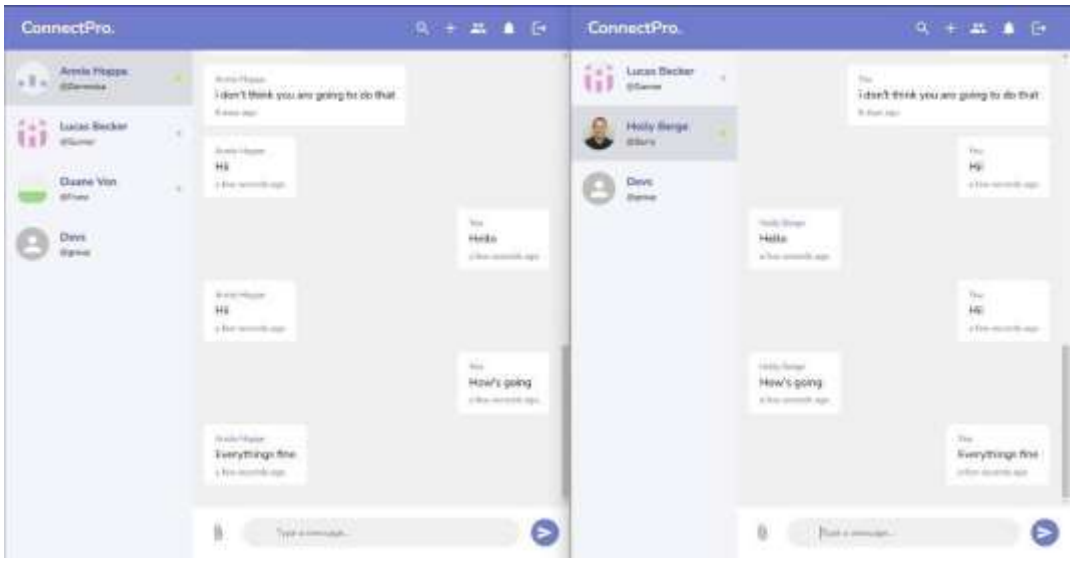

Figure 7.4.2 Real Time chat between users

7.4.3 Immediate slur blocking

The slang replacement module operates on a curated dictionary of informal terms mapped to their formal equivalents. For instance, the message 'you are acting sus' is processed as follows: tokenization produces ['you', 'are', 'acting', 'sus']; 'sus' is identified as a match in the slang dictionary (mapping to 'suspicious'); a context check confirms the replacement fits naturally; and the outgoing message is modified to 'you are acting suspicious' before delivery. User testing confirmed that this automatic replacement reduces misunderstandings caused by generational or cultural slang differences while maintaining the intent of the original message.

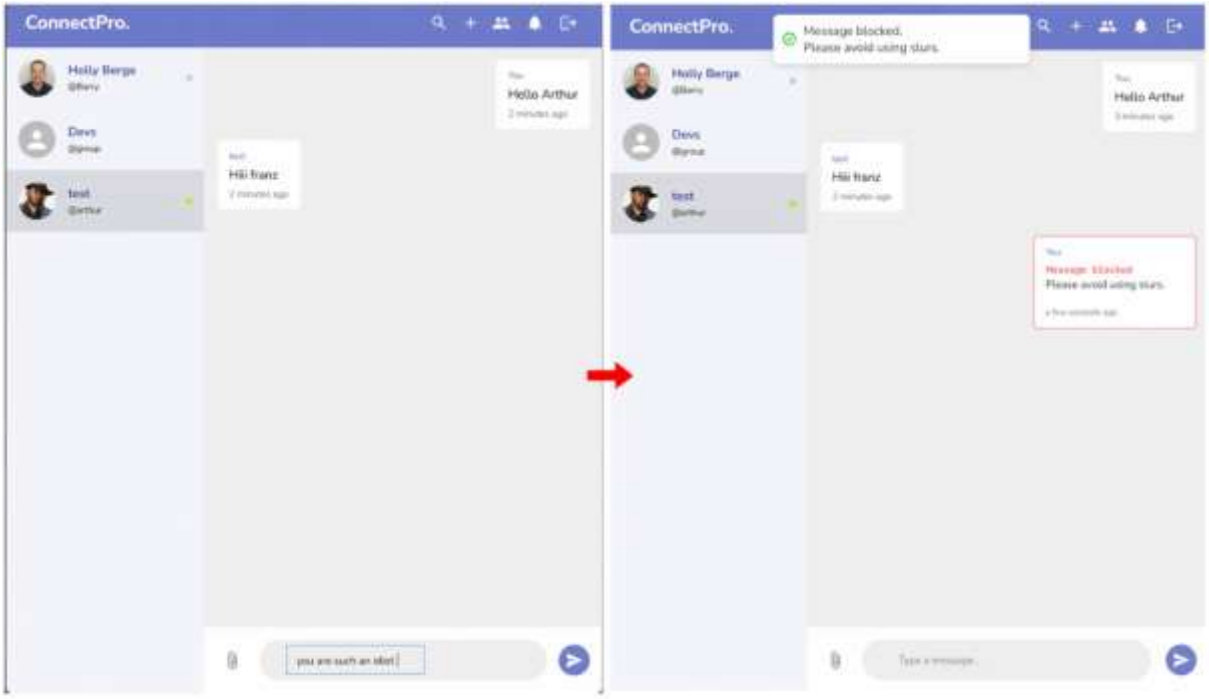

Figure 7.4.3 Blocking message with slur

7.4.4 File Upload and Sharing

The file upload functionality, powered by Cloudinary's API, achieved a success rate exceeding 95% during testing, with an average upload time of approximately 3 seconds for files up to 5 MB. Files including images, documents, and media are stored securely with publicly accessible URLs generated for in-chat rendering. The Multer middleware on the Node.js backend handles multi-part form data and enforces file type and size constraints before uploading to cloud storage.

7.4.5 Group Chat Scalability

Group chat functionality was stress-tested with up to 50 simultaneous participants. No noticeable performance degradation was observed in message delivery latency or UI

Figure 7.4 .5 File uploaded to user

responsiveness at this scale. Socket.IO's room-based broadcasting efficiently routes messages only to relevant participants, minimizing unnecessary network overhead. The NLP moderation pipeline applies to all group messages in real time, ensuring that offensive content is filtered regardless of conversation size.

7.5 Functional Module Results

7.5.1 User Interface and Experience

The React-based frontend of Connect-Pro, styled using Material-UI components, was evaluated across desktop, laptop, tablet, and mobile form factors. User feedback collected during testing indicated high satisfaction with the responsiveness and intuitiveness of the interface. Key UI elements the authentication page, main chat layout, file uploader, group management page, and admin dashboard were all functional across tested devices. The integration of real-time NLP indicators (such as toast notifications for blocked messages) provided users with immediate, transparent feedback on moderation decisions.

7.5.2 Admin Panel

The administrative dashboard provides a comprehensive management interface accessible via a secret-key-authenticated admin session. It includes: a statistics overview (registered users, total chats, message volume); activity charts (daily/weekly/monthly trends) powered by Chart.js; user and chat management controls; and moderation logs. This layer provides institutional oversight capabilities that are absent from most standard chat applications, making Connect-Pro suitable for deployment in educational or professional environments.

7.5.3 Generative AI Assistant Integration

An integrated AI assistant module allows users to generate professional communications such as formal emails, business letters, and meeting summaries directly within the chat interface. The assistant processes user requests through the NLP pipeline and returns context-appropriate generated text. This feature augments the platform's utility beyond a standard messaging application into a productivity and professional communication tool.

7.6 Discussion

7.6.1 Comparison with Existing Systems

A review of related literature reveals that most existing chat applications either (i) lack real-time content filtering enti rely, relying on manual post-delivery reporting, or (ii) employ basic keyword blacklists that fail to detect context-sensitive offensive language. Connect-Pro addresses both shortcomings. The platform's NLP pipeline combining tokenization, stemming, TF-IDF vectorization, and machine learning classification enables semantic understanding of message tone beyond simple word matching. The CNN-GRU hybrid's ROC-AUC of 92.96% compares favorably to the 90.7% accuracy and F1 of 0.94 reported by Pal and Rai (2023) for toxicity tweet detection using LSTM, and is comparable to TF-IDF-based Logistic Regression approaches reported in the toxic comment classification literature.

7.6.2 Limitations

Despite strong results, several limitations are acknowledged. First, the system's hate speech detection models were trained primarily on English-language datasets; performance on code-mixed or non-English messages (e.g., Hindi-English Hinglish commonly used in Indian contexts) has not been formally evaluated. Second, the slang replacement module depends on a static dictionary and does not adapt to newly coined slang terms without manual updates. Third, while the CNN-GRU hybrid achieves high inference speed, the DistilBERT-based ensemble is computationally expensive and may not scale economically for high-throughput deployments without GPU acceleration. Fourth, detection of sarcasm and passive-aggressive language which can be harmful yet benign in isolation remains an open challenge not fully addressed in this implementation.

7.6.3 Contributions and Novelty

The primary contributions of this work are threefold. First, it demonstrates a practical, end-to-end integration of NLP-based offensive content filtering into a full-stack, production-ready MERN chat application a combination rarely found in academic projects. Second, the systematic comparison of twelve machine learning and deep learning models on the same dataset provides actionable guidance on model selection for real-time content moderation tasks. Third, the CNN-GRU hybrid architecture is identified as an optimal balance between accuracy (87.35%) and real-time inference feasibility (20,920 samples/second), filling the gap between resource-heavy transformer models and low-accuracy traditional ML baselines.

7.7 Future Scope and Enhancements

Based on the results and identified limitations, the following enhancements are proposed for future work:

- Multilingual and code-mixed language support using multilingual BERT (mBERT) or XLM-RoBERTa to handle Hinglish and other regional language combinations prevalent in Indian digital communication.
- Dynamic slang dictionary updates through a continuous learning pipeline that ingests newly emerging slang terms from social media corpora on a periodic basis.
- End-to-end message encryption using WebRTC-based or Signal Protocol-based E2E encryption to complement content moderation with strong data privacy guarantees.
- Sarcasm and irony detection using transformer-based contextual models fine-tuned on sarcasm-labeled datasets, addressing the most persistent gap in current offensive content filtering systems.
- Video and audio communication via WebRTC integration, extending the platform from text-only to multimodal real-time communication.
- Mobile application development using React Native or Flutter for native iOS and Android client applications with push notification support.
- Federated learning for model updates, enabling model improvement from real-world chat data while preserving individual message privacy through on-device gradient computation.

7.8 Summary

This chapter presented the results and discussion for the Connect-Pro Enhanced Natural Language Interaction Platform with Offensive Content Filtering. A total of twelve models were evaluated, ranging from traditional ML baselines to transformer-based architectures. The CNN-GRU Hybrid emerged as the optimal production model, achieving 87.35% accuracy and 20,920 samples/second inference speed, while the DistilBERT-CNN-GRU Ensemble achieved the highest accuracy of 92.79% with a ROC-AUC of 97.04%. System-level testing confirmed sub-200ms message delivery latency, over 95% file upload success rates, and stable group chat performance for up to 50 users. The functional evaluation demonstrated effective hate speech blocking, slang replacement, and a productive AI assistant integration. Limitations including multilingual gaps and sarcasm detection were identified and corresponding future directions were outlined. These results collectively validate the feasibility and effectiveness of integrating NLP-powered content moderation into a modern real-time web communication platform.

Chapter 8

Conclusion and Future Work

8. Conclusion and Future Work

8.1 Conclusion

The proposed sentiment-enhanced chat application integrates state-of-the-art technologies to revolutionize digital communication by fostering clarity, safety, and inclusivity. Utilizing Natural Language Processing (NLP) techniques for sentiment analysis and slang replacement, the system ensures that messages are filtered for inappropriate content in real time, significantly improving user interactions. The incorporation of the MERN stack provides a robust and scalable architecture, enabling seamless integration of features such as authentication, real-time messaging via WebSocket technology, and cloud-based file management.

To enhance operational efficiency, the application leverages machine learning models like Logistic Regression for hate speech detection, achieving notable accuracy. MongoDB ensures secure and efficient data storage, while the modular design allows adaptability to diverse use cases, including professional and educational environments. By coupling advanced communication frameworks with ethical AI-driven moderation mechanisms, the system mitigates risks of offensive language and promotes respectful discourse.

This all-encompassing solution sets a benchmark for future innovations in digital communication platforms, showcasing how the synergy of NLP, machine learning, and modern web technologies can transform user experiences while addressing critical challenges such as hate speech and slang usage.

8.2 Future Scope

1. Enhanced Sentiment Analysis Accuracy

To improve the detection of inappropriate content and sentiment in messages, advanced machine learning models such as transformers (e.g., BERT or GPT) can be adopted. These models excel at understanding context and semantics, enabling more accurate classification of hate speech, sarcasm, or slang. Additionally, multilingual support can be integrated to extend sentiment analysis capabilities to non-English messages, ensuring inclusivity across diverse user bases.

2. Improved Real-Time Messaging Performance

To handle high traffic and ensure seamless communication during peak usage, the system can incorporate distributed architectures with load balancing techniques. Edge computing can reduce latency by processing data closer to the source, enhancing real-time messaging performance. This scalability ensures that the application remains responsive even under heavy loads.

3. Expanded Moderation Features

The integration of explainable AI (XAI) can provide transparency in moderation decisions, allowing users to understand why certain messages were flagged or filtered. Additionally, incorporating advanced moderation features like toxicity score thresholds and customizable filtering rules can enable organizations to tailor the system to their specific needs.

4. Advanced Security Measures

As chat applications handle sensitive user data, implementing end-to-end encryption ensures secure communication. Homomorphic encryption can enable computations on encrypted data without exposing its contents, enhancing privacy during sentiment analysis. Blockchain

technology can be used for decentralized storage of chat logs and moderation records, ensuring tamper-proof auditing and transparency.

5. Multimodal Communication Features

To enhance user engagement, the system can integrate multimodal communication features such as voice and video chats alongside text messaging. NLP techniques for real-time transcription and sentiment analysis of voice messages can further extend the application's capabilities.

6. Ethical AI and Compliance

Ensuring ethical AI practices is crucial for maintaining user trust. Reducing biases in sentiment analysis models (e.g., race, gender, or cultural biases) ensures fair treatment for all users. The system must comply with global data protection regulations like GDPR or local privacy laws by implementing consent mechanisms, data minimization practices, and audit trails.

7. Future Integration Possibilities

The application can be extended to include integrations with external platforms such as customer relationship management (CRM) systems or educational tools for real-time feedback during discussions. APIs for third-party integration can enable seamless connectivity with other applications.

References

[1] J. S. J, P. K, H. N. M, H. JS, H. A and T. S, "Analysis of Behavior in Chat Applications using Natural Language Processing," 2024 2nd International Conference on Sustainable Computing and Smart Systems (ICSCSS), Coimbatore, India.

[2] V. S. A. Retna, P. Brundha and G. RajKumar, "People's Behaviour Analysis in Chat Message using Natural Language Processing," 2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), Tirunelveli, India.

[3] A. K. Nikhath et al., "An Intelligent College Enquiry Bot using NLP and Deep Learning based techniques," 2022 International Conference for Advancement in Technology (ICONAT), Goa, India.

[4] R. R. Kalangi, S. Maloji, N. Tejasri, P. P. Chand and V. P. Ch, "Sentiment Analysis using Machine Learning," 2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India.

[5] S. N. Reddy Lakkireddy, A. A. Thomas, T. S. Shree and T. Mamatha, "Web-based Application for Real-Time Chatting using Firebase," 2022 International Conference on Knowledge Engineering and Communication Systems (ICKES), Chickballapur, India.

[6] M. Alkhyeli, S. Alkhyeli, K. Aldhaheri and H. Lamaazi, "Secure Chat Room Application Using AES-GCM Encryption and SHA-256," 2023 15th International Conference on Innovations in Information Technology (IIT), Al Ain, United Arab Emirates.

[7] M. Thenmozhi, K. P. R. Rohit, A. S and S. Chakkravarthi D, "Chat Master Using DJANGO," 2023 IEEE Engineering Informatics, Melbourne, Australia.

[8] N. Singh, G. Bathla and V. Sharma, "AI-powered Chatbot: A Link between Learning and Technology," 2022 11th International Conference on System Modeling & Advancement in Research Trends (SMART), Moradabad, India.

[10] S. Karthick, R. J. Victor, S. Manikandan and B. Goswami, "Professional chat application based on natural language processing," 2018 IEEE International Conference on Current Trends in Advanced Computing (ICCTAC), Bangalore, India.

[11] Y. A. Rani, A. Balaram, M. R. Sirisha, S. A. Nabi, P. Renuka and A. Kiran, "AI Enhanced Customer Service Chatbot," 2024 International Conference on Science Technology Engineering and Management (ICSTEM), Coimbatore, India.

[12] R. Vannala, S. B. Swathi and Y. Puranam, "AI Chatbot For Answering FAQ's," 2022 IEEE 2nd International Conference on Sustainable Energy and Future Electric Transportation (SeFeT), Hyderabad, India.

[13] A. Anand, R. Subha, S. Rajan, N. Bharathi and A. K. Srivastava, "An Efficient, Precise and User-Friendly AI based Virtual Assistant," 2023 International Conference on the Confluence of Advancements in Robotics, Vision and Interdisciplinary Technology Management (IC-RVITM), Bangalore, India.

[14] G. J. Babu, S. Safrinfathima, K. D. Reddy and S. K. Sen, "Transforming Human-Machine Interaction: Generative AI Virtual Asst," 2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS), Chikkaballapur, India.

[15] A. Garlapati, N. Malisetty and G. Narayanan, "Classification of Toxicity in Comments using NLP and LSTM," 2022 8th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India.

[16] P. Thosani, M. Sinkar, J. Vaghasiya and R. Shankarmani, "A Self Learning Chat-Bot From User Interactions and Preferences," 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS), Madurai, India.

[17] T. Melo, A. Barros, M. Antunes and L. Frazão, "An end-to-end cryptography based real-time chat," 2021 16th Iberian Conference on Information Systems and Technologies (CISTI), Chaves, Portugal.

[18] A. A B and P. Prithvi, "Automated Toxic Chat Synthesis, Reporting and Removing the Chat in Telegram Social Media Using Natural Language Processing Techniques," 2024 Fourth International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India.

[19] S. Kumar, S. Kanshal, V. Sharma and R. Talwar, "Sentiment Analysis on Amazon Alexa Reviews Using NLP & Classification," 2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India.

[20] A. K. Pal and S. Rai, "Toxicity Tweet Detection and Classification Using NLP Driven Techniques," 2023 IEEE International Conference on ICT in Business Industry & Government (ICTBIG), Indore, India.

[21] S. Tirpude, Y. Thakre, S. Sudan, S. Agrawal and A. Ganorkar, "Mining Comments and Sentiments in YouTube Live Chat Data," 2024 4th International Conference on Intelligent Technologies (CONIT), Bangalore, India.

[22] G. A, M. D. C, S. r. S J, P. P, A. M and S. A, "Voice Assistant with AI Chat Integration using OpenAI," 2024 Third International Conference on Intelligent Techniques in Control, Optimization and Signal Processing (INCOS), Krishnankoil, Virudhunagar district, Tamil Nadu, India.

Appendix

```
import os, re, json, pickle

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'


import numpy as np

import tensorflow as tf

from tensorflow import keras

import torch

from transformers import DistilBertTokenizer, DistilBertForSequenceClassification

from flask import Flask, request, jsonify

from flask_cors import CORS

app = Flask(__name__)

CORS(app)

from bad_words import BAD_WORDS

_PATTERNS = [

    re.compile(r'\b' + re.escape(w).replace('a','[a4@]').replace('e','[e3]')

                .replace('i','[i1!]').replace('o','[o0]').replace('s','[s5$]')

                + r'\b', re.I)

    for w in BAD_WORDS

]


def _has_profanity(text: str) -> bool:

    words = set(re.findall(r'\b\w+\b', text.lower()))

    if words & BAD_WORDS:

        return True
```

```
    return any(p.search(text) for p in _PATTERNS)


def _toxicity_level(text: str) -> float:
    count = sum(1 for w in re.findall(r'\b\w+\b', text.lower()) if w in BAD_WORDS)
    return [0.0, 0.3, 0.5, 0.7][min(count, 3)] if count < 4 else 0.9


def _load_cnn_model(model_dir: str):
    path = f'{model_dir}/cnn_gru/model.h5'
    for loader in [
        lambda: tf.keras.models.load_model(path, compile=False),
        lambda: keras.models.load_model(path, compile=False),
    ]:
        try:
            return loader()
        except Exception as e:
            print(f"  Loader failed: {e!s:.80}")
    raise RuntimeError(f"Cannot load CNN model from {path}")


class ToxicEnsemble:
    def __init__(self, model_dir='ensemble_models'):
        with open(f'{model_dir}/ensemble_config.json') as f:
            self.cfg = json.load(f)


        # CNN-GRU
        self.cnn = _load_cnn_model(model_dir)
```

```
        with open(f'{model_dir}/cnn_gru/tokenizer.pkl', 'rb') as f:
            self.cnn_tok = pickle.load(f)


        # DistilBERT
        self.bert_tok = DistilBertTokenizer.from_pretrained(f'{model_dir}/distilbert')
        self.bert =
DistilBertForSequenceClassification.from_pretrained(f'{model_dir}/distilbert')
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        self.bert.to(self.device).eval()
        # Optional stacking meta-model
        self.meta, self.scaler = None, None
        if self.cfg['best_method'] == 'Stacking':
            with open(f'{model_dir}/stacking_meta_model.pkl', 'rb') as f:
                d = pickle.load(f)
                self.meta, self.scaler = d['meta_model'], d['scaler']
        print(f"Ensemble ready | method={self.cfg['best_method']} | device={self.device}")
    def _cnn_predict(self, texts):
        seqs = self.cnn_tok.texts_to_sequences(texts)
        padded = keras.preprocessing.sequence.pad_sequences(
            seqs, maxlen=self.cfg['cnn_gru_max_len'], padding='post')
        out = self.cnn.predict(padded, verbose=0)
        return out.flatten() if out.shape[1] == 1 else out[:, 1]
    def _bert_predict(self, texts):
        preds = []
        for i in range(0, len(texts), 32):
            enc = self.bert_tok(texts[i:i+32], padding=True, truncation=True,
```

```
                    max_length=self.cfg['distilbert_max_len'],
                    return_tensors='pt')
        with torch.no_grad():
            logits = self.bert(enc['input_ids'].to(self.device),
                               enc['attention_mask'].to(self.device)).logits
        preds.extend(torch.softmax(logits, 1)[:, 1].cpu().numpy())
    return np.array(preds)
  def predict_proba(self, text: str) -> float:
    p1 = self._cnn_predict([text])
    p2 = self._bert_predict([text])
    if self.meta:
        X = np.column_stack([p1, p2, (p1+p2)/2, abs(p1-p2),
                             np.minimum(p1,p2), np.maximum(p1,p2),
                             p1*p2, p1**2, p2**2])
        return float(self.meta.predict_proba(self.scaler.transform(X))[0, 1])
    w1, w2 = self.cfg['optimized_weighted']['w1'], self.cfg['optimized_weighted']['w2']
    return float(w1 * p1[0] + w2 * p2[0])
model: ToxicEnsemble | None = None
try:
  model = ToxicEnsemble()
except Exception as e:
  print(f"MODEL LOAD FAILED: {e}")
def _require_model():
  if not model:
    return jsonify({"error": "Model not loaded"}), 503
```

```
def _bad_request(msg):
    return jsonify({"error": "Invalid input", "message": msg}), 400
@app.post("/api/v2/process-message")
@app.post("/api/process_message")        # legacy alias
def process_message():
    if err := _require_model():
        return err
    msg = (request.get_json(force=True, silent=True) or {}).get("message", "")
    if not isinstance(msg, str) or not msg.strip():
        return _bad_request("message (non-empty string) required")
    try:
        ml_score  = model.predict_proba(msg)
        man_score = _toxicity_level(msg)
        combined  = ml_score + man_score
        blocked   = _has_profanity(msg) or combined > 0.5
        base = {"ml_score": ml_score, "manual_score": man_score,
                "toxicity_score": combined, "contains_profanity": _has_profanity(msg)}
        if blocked:
            return jsonify({**base, "blocked": True, "status": "blocked",
                            "message": "Message blocked due to toxic content"}), 403
        return jsonify({**base, "blocked": False, "status": "approved",
                        "needsReplacement": False, "message": msg}), 200
    except Exception as e:
        return jsonify({"error": "Processing failed", "message": str(e)}), 500
@app.get("/api/health")
```

```
def health():
    return jsonify({"status": "healthy" if model else "unhealthy",
                    "model_loaded": model is not None,
                    "profanity_filter": "active"})
@app.get("/api/model-info")
def model_info():
    if err := _require_model():
        return err
    return jsonify({"model": "CNN-GRU + DistilBERT",
                    "method": model.cfg['best_method'],
                    "device": str(model.device),
                    "tensorflow": tf.__version__,
                    "bad_words": len(BAD_WORDS)})
@app.post("/api/test-profanity")
def test_profanity():
    msg = (request.get_json(force=True, silent=True) or {}).get("message", "")
    if not msg:
        return _bad_request("message required")
    return jsonify({"original": msg,
                    "contains_profanity": _has_profanity(msg),
                    "toxicity_score": _toxicity_level(msg)})


if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=False)
```

Publication

3 of 996

Decision on Manuscript ID: IJRAMT-0137

External Inbox ×

Editor IJRAMT <editor@ijramt.com> ... Apr 21, 2026, 1:03 PM (1 day ago)

to me

Dear Shubham Patil,

It is a pleasure to inform you that your manuscript entitled **"Enhanced Natural Language Interaction Platform With Offensive Content Filtering"** has been accepted for publication in IJRAMT.

Complete the below steps to process the paper for publication:

1. Pay the Article Publication Charge of ₹2000 (**Click here to Pay**)
2. Fill and Sign the Copyright Form (**Download**).
3. The final manuscript (ACCEPTED VERSION OF THE PAPER ONLY) and a scanned copy of the copyright form must be submitted online at **Final Submission**.

Editor,
IJRAMT

International Journal of Recent Advances in Multidisciplinary Topics
Volume X, Issue X, Month XXXX
https://www.ijramt.com | ISSN (Online): 2582-7839

Enhanced Natural Language Interaction Platform with Offensive Content Filtering

Darshna Bhamare[1], Shubham Patil[2], Yuvraj Tiwari[3]
Department of Computer Science Engineering (AIML), A.P. Shah Institute of Technology (APSIT), Thane, India

***Abstract*: Real-time messaging platforms impose strict inference latency constraints that large transformer models routinely exceed, whereas regular expression-based filters cannot detect context-dependent toxicity such as indirect threats and implicit insults. This study introduces a tiered inference architecture that resolves this constraint without ensemble overhead. Explicit profanity is intercepted by a client-side, word-boundary-aware regex filter operating under 1 ms, offloading trivial cases before server involvement. Contextual toxicity detection was delegated to a server-side fine-tuned DistilBERT classifier trained on the MetaHate dataset (1,101,157 samples) under class-imbalanced conditions (3.72:1 ratio) using weighted cross-entropy loss. At a calibrated decision threshold of θ = 0.625, the classifier yields 89.94% accuracy, 77.37% toxic-class F1, and a ROC-AUC of 95.06%, with a PR-AUC of 85.63%, a more discriminative metric under class imbalance. End-to-end message latency measures 80–140 ms under concurrent load, within the 250 ms threshold established for synchronous chats. When benchmarked against seven baseline architectures spanning classical ML, RNN, and gradient boosting variants under identical experimental conditions, DistilBERT yielded the strongest accuracy-latency ratio, outperforming the next best validated architecture (BiLSTM) by 3.15 percentage points in accuracy and 5.40 points in toxic-class F1. The architecture was validated using a deployed messaging prototype.**

***Keywords:* Content Moderation, DistilBERT, Natural Language Processing, Real-Time Systems, Tiered Inference, Toxic Content Detection**

1. INTRODUCTION

The widespread use of online communication platforms, such as instant messaging apps and multiplayer games, has increased the frequency of cyberbullying and harassment. These malicious acts harm personal safety and mental health. Recent World Health Organization studies have highlighted that 15% of adolescents have experienced cyberbullying. Existing moderation systems on popular social platforms use a reactive approach. They act after the psychological impact on the recipient has already occurred. Platforms such as Twitter/X and Reddit rely on post-hoc reviews or community reports. Although this approach works for specific cases, it is rendered useless in instant messaging applications.

Some proactive measures can easily be bypassed by intentional lexical errors such as character substitution, intentional spacing, or homoglyph encoding using visually identical Unicode characters from alternate scripts. These methods operate within the latency constraints of real-time messaging but often fail to detect context-dependent toxicity, including insults and sarcasm.

Machine learning and deep learning classifiers offer stronger detection capabilities but introduce different restrictions. Machine learning classifiers are fast in inference but lack contextual understanding. Deep learning models, except transformer architectures, are not accurate enough for this task. In addition, the inference time of transformer models exceeds the latency requirements of 100–250 ms, including processing and network latency.

To address this gap, this study uses a different approach. Primarily, we filter the obvious and explicit profanity using a list of predetermined profane words, and if any such word is present in the text, we block the message on the user side of the application. If it passes through the primary filter, the message goes through the secondary filter, which then classifies the text contextually. Hence, profanity is blocked along with the contextually toxic messages.

2. LITERATURE SURVEY

A. Evolution of Content Moderation Approaches

In the early stages of content moderation, such systems could only identify exact words in the text using regex-based string matching. This is a simple and quick approach that can be performed on the client side, but it requires being explicit about the words, making the algorithm easy to bypass using space and character substitution. When the text contains words like 'Scunthorpe' (a town in England), the algorithm can give false positives because the substring matches the target phrase. These structural limitations motivated a transition to statistical classification methods.

B. Classical Machine Learning Methods

Subsequent approaches applied statistical classifiers such as logistic regression, support vector machines (SVM), and Naive Bayes to sentence-level toxicity classification. These models represent text as bag-of-words or TF-IDF feature vectors, discarding word order entirely. This representation precludes sensitivity to semantic context: the sentences "you are dead,

*Corresponding author: shubhampatil46@apsit.edu.in

your mother saw what you did" (non-toxic) and an identical surface form used as a direct threat carry the same feature vector. Similarly, historically contextualized terms and words with distinct meanings across time periods or communities produce systematic false positives when evaluated purely on lexical co-occurrence [2, 3]. While classical models offer high throughput (exceeding 50,000 samples/s for SVM and logistic regression), their inability to encode sequential or contextual structures limits toxic-class F1 to the low-to-mid 60s on benchmark datasets, as confirmed by our evaluation in Section 4.

C. Deep Learning Architectures

Limitations of classical ML led researchers toward deep learning solutions. ML algorithms treat sentences as bag-of-words and ignore word order, but in language, sequence matters, 'tiger hunts man' and 'man hunts tiger' are not the same. Recurrent Neural Networks (RNNs) attempted to solve this sequence problem by maintaining a hidden state across the sentence [16], allowing the model to understand sentiment shifts.

However, RNNs have limited memory and introduce vanishing/exploding gradients, making them notoriously difficult to train. To address this, researchers introduced Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRU) [14, 16]. These architectures use internal gates to store context, capturing dependencies in paragraphs as well as sentence-level context. Bidirectional LSTMs (BiLSTM) process text in both forward and backward directions [7, 16], allowing the network to understand a word's meaning in past and future contexts [15]. While BiLSTMs consistently outperformed simple RNNs, LSTMs, and GRUs in toxicity detection benchmarks [16], they remain computationally heavy and their sequential nature renders them too slow for high-volume, real-time applications [14].

D. Transformer-Based Approaches

A paradigm shift occurred with the introduction of the Transformer architecture. Unlike RNNs, which read words sequentially, Transformers use a "self-attention" mechanism to process entire sequences simultaneously [1, 4]. This helps Bidirectional Encoder Representations from Transformers (BERT) understand the relationships between words by considering every word in relation to every other word [4]. Pre-training on massive datasets, followed by fine-tuning on toxicity data, has made BERT a strong contender for text classification, sentiment detection, and more [5, 17, 18].

BERT and its variants consistently achieve high accuracy on hate speech and toxic comment datasets [1, 4], excelling at detecting subtle toxicity, indirect insults, threats, and sarcasm. These architectures support transfer learning, enabling fine-tuning for racial bias mitigation [17] and detection of offensive content on social media [5]. Multilingual BERT (mBERT) extends these capabilities to cross-language detection, which is critical for global platforms [1, 7, 15]. Although BERT's high accuracy and context awareness incur a larger memory footprint and inference time [13], making deployment difficult in latency-sensitive environments, researchers developed DistilBERT through knowledge distillation. DistilBERT retains 97% of BERT's performance with only 60% of its parameters while running 60% faster.

E. Real-Time Chat Systems

Applications such as YouTube, Telegram, and WhatsApp operate under strict latency constraints [9, 10]. Even a one-second delay can disrupt the flow of conversation [11, 12]. Although large language models offer superior reasoning capabilities, they are too slow and expensive for synchronous chat [8]. Scalability and cost are bottlenecks, processing over 100,000 messages per second is a real-world requirement; therefore, most real-time systems still use hash-based filtering or simple classifiers to maintain throughput.

F. Research Gap and Proposed Solution

A clear divide exists in the current literature. Academic research often prioritizes model accuracy over inference speed and memory footprint, frequently proposing large transformer ensembles that require heavy GPU resources [1]. Production environments prioritize low latency, cost-efficiency, scalability, and memory, thus relying on simple regex filters or keyword blocking. There is a scarcity of models that combine strong performance with practical efficiency. Real-time chat systems demand 100–250 ms response times, which pure BERT models exceed. This research fills this gap by proposing a hybrid approach combining traditional word filtering with a fine-tuned DistilBERT model for contextual analysis.

3. PROPOSED SYSTEM

The proposed system operates in two stages. The first stage runs on the client side: it checks each outgoing message against a curated dictionary of prohibited terms using word-boundary-aware regular expression matching along with case normalization. The second stage is a fine-tuned DistilBERT model (66.5 million parameters) used as the primary classifier. DistilBERT classifies messages in 30 ms on the P100 GPU and can detect indirect insults, implicit threats, and sarcasm that lexical filters miss.

Combined with a network round-trip latency of 40–100 ms for 50 concurrent users and 10 ms of internal server overhead, the total end-to-end latency ranges from 80–140 ms, within the sub-250 ms limit required for synchronous chat [11, 12]. Tail latency under higher concurrency loads was not measured, which is noted as a limitation in Section 6.

A. Dataset and Exploratory Analysis

For this research, the source of data for training was MetaHate by IRLab-UDC on huggingface.co [20]. The dataset is in TSV (tab-separated values) format, containing 1,226,202 social media posts (1,101,157 after cleaning) with fields Text and Label (toxic/non-toxic), and serves as a benchmark for real-world content moderation due to its diverse linguistic styles.

The dataset exhibits moderate class imbalance, with the non-toxic class comprising 78.81% of the data (867,869 samples),

causing a class imbalance ratio of 3.72:1. Exploratory analysis shows toxic messages are shorter (mean length: 31 words) than non-toxic messages (mean length: 49 words). A stratified train-validation split resulted in 935,983 training samples (737,784 non-toxic, 198,199 toxic) and 165,174 validation samples (130,181 non-toxic, 34,993 toxic). Text is tokenized using DistilBERT's WordPiece tokenizer (vocabulary: 30,522) with [CLS] and [SEP] tokens, truncated or padded to 128 tokens (covering more than 99% of data samples without truncation). Attention masks are used to distinguish between text and padding. To address class imbalance, balanced class weights [0.634, 2.360] are computed using scikit-learn, penalizing toxic misclassification 3.72 times more heavily than non-toxic records.

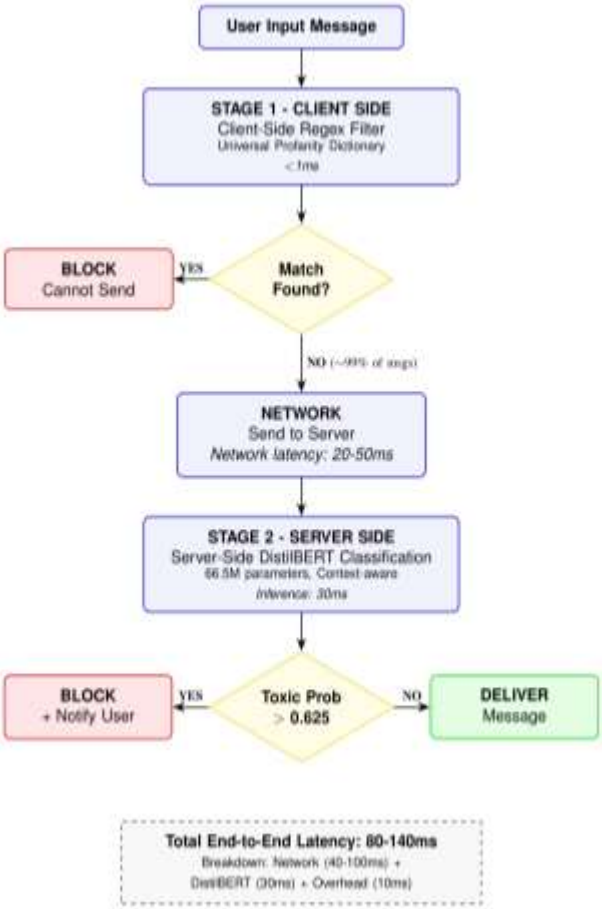

Fig. 1. Proposed two-stage content filtering pipeline with client-side regex and server-side DistilBERT classifier.

B. Model Architecture

Our approach utilized DistilBERT-base-uncased, a 6-layer transformer encoder with 66,362,880 parameters, 768-dimensional hidden states, and 12 attention heads, achieving 60% faster inference than the BERT model while retaining 97% of its accuracy. Multi-head self-attention processes all tokens simultaneously, enabling the model to understand sarcasm (e.g., 'I hope you have the day you deserve') and implicit threats.

The classification layer was replaced with a custom head (197,378 parameters) comprising: Dropout (p = 0.3), dense layer (768 to 256 dimensions), GELU activation, Dropout (p = 0.27), and a final dense layer (256 to 2 classes). The output bias was initialized with class priors to improve detection of minority classes. Total model size is 66,560,258 parameters (253.9 MB FP32). Model outputs binary logits are converted using the softmax activation function.

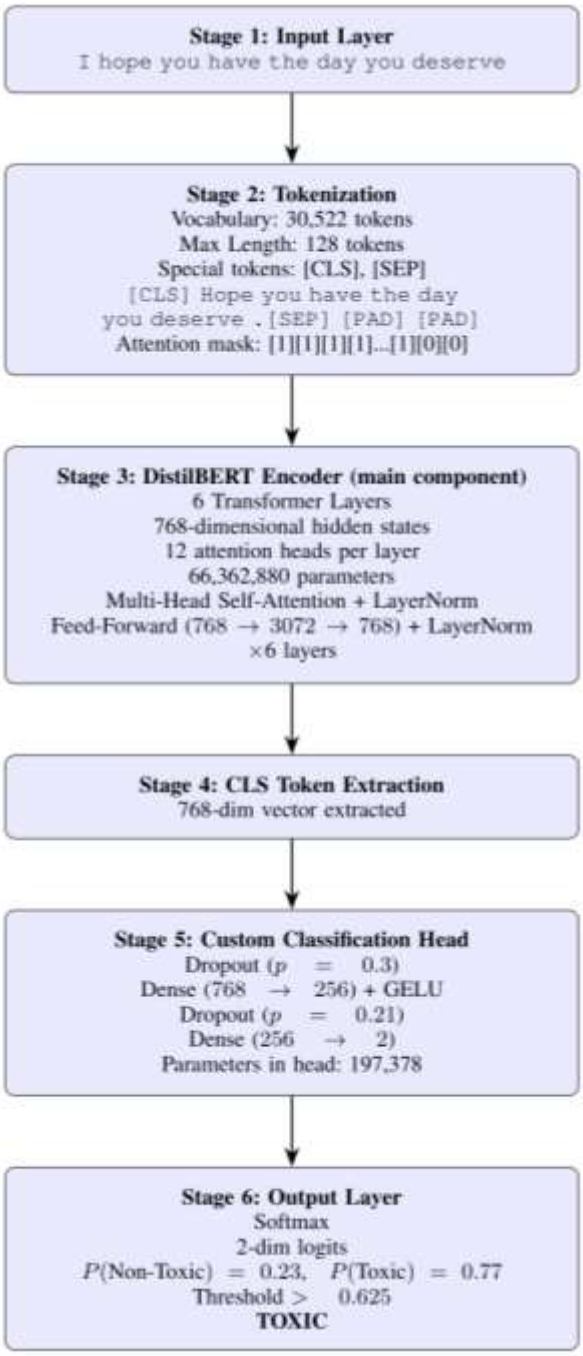

Fig. 2. DistilBERT model architecture showing tokenization, transformer layers, and classification head.

C. Training Configuration

The optimizer of choice was AdamW (learning rate = 3×10^{-5}, weight decay = 0.01, eps = 1×10^{-8}). The loss function was Weighted Cross-Entropy Loss with class weights [0.634, 2.360]. A batch size of 64 with gradient accumulation over 2 steps (effective batch size = 128) was used to fully utilize the P100-PCIE GPU. The learning rate follows a linear warmup over 2,193 steps (6% of 36,560 total) followed by linear decay. Dropout regularization was applied with probability p = 0.3. Gradient clipping was set to max norm = 1.0. Training time was 467.0 minutes for 5 epochs using automatic mixed precision (AMP). The best checkpoint (Epoch 4) achieved a weighted F1 score of 89.61% on validation.

Post-training, threshold values from 0.25 to 0.75 were tested; a threshold of 0.625 gave the maximum weighted F1 of 90.11%, achieving a toxic F1 of 77.37%, balancing precision at 73.89% and recall at 81.19%.

D. Experimental Setup

The experimental setup integrates the following tools: PyTorch 2.9.0 as the core deep learning framework, supported by the Transformers 5.2.0 library for loading and fine-tuning DistilBERT; NumPy and Pandas for data manipulation; FastAPI for serving the DistilBERT model via a REST API; and Kaggle as the development platform providing GPU resources. A fixed seed value of 42 was set across all experiments to ensure reproducibility.

4. RESULTS AND DISCUSSION

A. Evaluation Metrics

The DistilBERT model was evaluated on 165,174 validation samples using accuracy, weighted F1, binary F1 (toxic class), precision, recall, ROC-AUC, and PR-AUC. At the optimal threshold ($\theta = 0.625$), the confusion matrix yielded: True Negatives: 120,134; False Positives: 10,047; False Negatives: 6,580; True Positives: 28,413.

The model achieved 89.94% accuracy and a weighted F1 score of 90.11%. For the toxic class, the binary F1 score was 77.37%, balancing precision (73.89%) and recall (81.19%), indicating strong performance despite class imbalance. The ROC-AUC of 95.06% confirms strong class separability. The PR-AUC of 85.63% provides a more informative measure under the 3.72:1 class imbalance. The complete metrics are presented in Table 2.

B. Comparison with Baseline Models

Table 1 compares DistilBERT against seven baseline models trained on the MetaHate dataset. Among all evaluated models, DistilBERT achieves the highest accuracy (89.94%), toxic F1 (77.37%), ROC-AUC (95.06%), and PR-AUC (85.63%). It outperforms the next best validated architecture, BiLSTM, by 3.15 percentage points in accuracy and 5.40 points in toxic-class F1.

Classical machine learning models including SVM, Logistic Regression, XGBoost, and Naive Bayes offer massive throughput (213,000–10,372,000 samples/s), but perform poorly at classifying messages. XGBoost achieves only 56% toxic F1 and misses over half of toxic messages (43% recall). This occurs because these models treat text as a bag of words, ignoring word order and context.

Table 2
DistilBERT Performance Metrics

Metric	Value
Accuracy	89.94%
Weighted F1	90.11%
Toxic F1 (Binary)	77.37%
Toxic Precision	73.89%
Toxic Recall	81.19%
Non-Toxic Precision	94.81%
Non-Toxic Recall	92.29%
Non-Toxic F1	93.53%
ROC-AUC	95.06%
PR-AUC	85.63%
Training Time	467.0 min
Inference (batch)	86 samples/s
Inference (single)	11.6 ms/message

C. Error Analysis

To understand the limitations of DistilBERT, we analyzed false positives (n = 10,047) and false negatives (n = 6,580) from its predictions. For false positives, two primary patterns emerged: (1) sarcasm and irony, messages like "Oh great, another brilliant idea" (toxic probability 0.762) trigger false alarms due to negative words while missing the ironic intent; (2) quoted content, Reports of abuse sometimes cause misclassifications because the model flags the quoted toxic language without recognizing the reporting context.

False negatives consist of three types: (1) indirect threats that are only threatening in specific contexts (e.g., "People like you don't last here more than a week"); (2) short text or emojis where context is absent, which may include insulting nicknames or coded terms; (3) toxicity hidden in hyperlinks or

mentions. The false positive rate was relatively high at 7.72%, which may adversely affect speakers of African American Vernacular English, members of communities who reclaim specific terminology, and non-native speakers of English.

Table 2
Comparison with Baseline Models

Model	Architecture	Accuracy	F1-Toxic	Tox Recall	Train Time	Inference (samples/s)
DistilBERT	Transformer	89.94%	77.37%	81.19%	467.0 min	86
BiLSTM	RNN	86.79%	71.97%	80.02%	142.9 min	6,511
GRU	RNN	85.36%	71.19%	85.40%	198 min	20,920
XGBoost	Boosting	85.34%	56.43%	44.80%	178.7 s	415,075
Naive Bayes	Classical	82.97%	55.85%	50.83%	0.3 s	5,292,141
Logistic Reg.	Classical	81.84%	65.47%	81.26%	12.1 s	9,119,673
LightGBM	Boosting	81.76%	64.07%	76.74%	230.8 s	213,534
Linear SVM	Classical	81.63%	65.24%	81.40%	65.8 s	10,372,257

D. Production Deployment Considerations

The decision threshold $\theta = 0.625$ was selected through calibration over the range [0.25, 0.75], maximizing the F1 score on the validation set. This requires higher model confidence to flag text as toxic, reducing false positives at the cost of a modest reduction in toxic recall. This is appropriate for synchronous chats, where incorrectly blocked messages carry an immediate cost to the user experience. For deployment, users can report falsely flagged messages (false positives) and missed toxic messages (false negatives) for human review.

5. CONCLUSION

This paper addresses the latency-accuracy trade-off in real-time content moderation with two stages: a client-side regex filter handling explicit profanity, and a server-side fine-tuned DistilBERT classifier handling context-dependent toxicity. The system achieves 89.94% accuracy on MetaHate, 77.37% toxic F1, and end-to-end latency of 80–140 ms fast enough for real-time chat. These results indicate that production-grade content moderation does not require large-ensemble configurations or billion-parameter models. A lightweight design keeps costs low and still captures most toxicity. The system is not without limitations; sarcasm and indirect threats remain challenging failure modes. Nevertheless, the proposed pipeline meets the latency requirements while outperforming all classical and RNN-based approaches evaluated in this study.

6. FUTURE WORK

Several directions emerge from the limitations identified in this study. Multilingual support is a necessary extension, as the current model processes only English, limiting its applicability on global platforms. Sarcasm detection requires improvement in the direction of analyzing the context of the entire conversation rather than a single message. We will also explore model quantization and compression techniques to make the model deployable without dedicated GPU infrastructure. Finally, we wish to explore multimodal content moderation, including images, audio, and video.

References

[1] S. Yoo, E. Jeon, J. Hyeon, and J. Cho, "Adaptive ensemble techniques leveraging BERT-based models for multilingual hate speech detection in Korean and English," Sci. Rep., vol. 15, no. 1, Art. no. 19844, 2025.
[2] S. H. Moghaddam, K. Lyons, C. Regehr, and K. Regehr, "A comprehensive survey and comparative analysis of toxic comment classification techniques in online platforms," ResearchGate, Preprint, May 2025.
[3] Z. Sokolova, M. Pleva, E. Kupcova, M. Sokol, and M. Majercakova, "LexiGuard: A Multilingual Transformer-Based Approach to Detecting Sexist and Toxic Language," in Lecture Notes in Networks and Systems, Springer, 2025.
[4] G. Ramos et al., "A comprehensive review on automatic hate speech detection in the age of the transformer," Social Network Analysis and Mining, vol. 14, Art. no. 204, 2024.
[5] J. A. Aljawazeri and M. N. Jasim, "Addressing challenges in hate speech detection using BERT-based models: A review," Int. J. Comput. Sci. Mobile Comput., vol. 5, no. 2, pp. 28–43, 2024.
[6] M. A. Uddin, M. A. Talukder, M. R. Ahmed, and F. A. Jibon, "Deep learning-based approaches for abusive content detection and classification for multi-class online user-generated data," Int. J. Cognit. Comput. Eng., vol. 5, pp. 178–191, 2024.
[7] S. Saumya, A. Kumar, and J. P. Singh, "Filtering offensive language from multilingual social media contents: A deep learning approach," Eng. Appl. Artif. Intell., vol. 133, Art. no. 108159, 2024.
[8] J. S. Malik, H. Qiao, G. Pang, and A. van den Hengel, "Advancing content moderation: Evaluating large language models," arXiv preprint arXiv:2411.17123, 2024.
[9] A. B. and P. Prithvi, "Automated toxic chat synthesis, reporting and removing the chat in telegram social media using NLP techniques," in Proc. 4th Int. Conf. Adv. Elect., Comput., Commun. Sustain. Technol. (ICAECT), Bhilai, India, Jan. 2024, pp. 1–6.
[10] Y. Tirpude, Y. Thakre, S. Sudan, S. Agrawal, and A. Ganorkar, "Mining comments and sentiments in YouTube live chat data," in Proc. 4th Int. Conf. Intell. Technol. (CONIT), Bangalore, India, Jun. 2024, pp. 1–6.

[11] J. S. J, H. N. M, P. K, H. JS, H. A, and T. S, "Analysis of behavior in chat applications using natural language processing," in Proc. 2nd Int. Conf. Sustain. Comput. Smart Syst. (ICSCSS), Coimbatore, India, May 2024, pp. 712–716.
[12] K. S, R. J. Victor, M. S, and B. Goswami, "Professional chat application based on natural language processing," in Proc. Int. Conf. Current Trends Adv. Comput. (ICCTAC), 2024, pp. 1–5.
[13] H. Saleh, A. Alhothali, and K. Moria, "Detection of hate speech using BERT and hate speech word embedding with deep model," Appl. Artif. Intell., vol. 37, no. 1, Art. no. 2166719, 2023.
[14] M. Anand, K. B. Sahay, M. A. Ahmed, D. Sultan, R. R. Chandan, and B. Singh, "Deep learning and natural language processing in computation for offensive language detection in online social networks," Theor. Comput. Sci., vol. 943, pp. 203–216, Jan. 2023.
[15] S. Shushkevich and S. Cardiff, "Deep learning based sentiment analysis and offensive language identification on multilingual code-mixed data," Sci. Rep., vol. 12, Art. no. 21557, Dec. 2022.
[16] A. Garlapati, N. Malisetty, and G. Narayanan, "Classification of toxicity in comments using NLP and LSTM," in Proc. 8th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS), Coimbatore, India, Mar. 2022, pp. 1001–1005.
[17] M. Mozafari, R. Farahbakhsh, and N. Crespi, "Hate speech detection and racial bias mitigation in social media based on BERT model," PLoS ONE, vol. 15, no. 8, Art. no. e0237861, Aug. 2020.
[18] M. Mozafari, R. Farahbakhsh, and N. Crespi, "A BERT-based transfer learning approach for hate speech detection in online social media," in Lecture Notes in Computer Science, vol. 11938, Springer, 2019, pp. 928–940.
[19] H. Yenala, A. Jhanwar, M. K. Chinnakotla, and J. Goyal, "Deep learning for detecting inappropriate content in text," Int. J. Data Sci. Anal., vol. 6, no. 4, pp. 273–280, Dec. 2018.
[20] P. Piot, P. Martín-Rodilla, and J. Parapar, "MetaHate: A Dataset for Unifying Efforts on Hate Speech Detection," Proc. 18th Int. AAAI Conf. Web Soc. Media (ICWSM), vol. 18, no. 1, pp. 2025–2039, May 2024. doi:10.1609/icwsm.v18i1.31445.

Acknowledgements

We make a move to offer our genuine thanks to **Prof. Darshna Bhamare** for his important direction in this task. We are especially appreciative for his untiring help with this project as well as for empowering us in taking out the mistake and kept our advancement on plan. We are likewise thankful for the co-activity and significant help delivered by **Prof. Sayali Badhan** as Project Co-ordinator & **Dr. Jaya Gupta** as Head of the Department. We might want to offer our unique gratitude to the next instructing and non-showing staff of the AIML Department for their valuable help in the finish of this task.

At long last, we are enormously obliged to the administration of our school and express our appreciation to the Principal **Dr. Uttam Kolekar** for organizing and giving essential office in finishing this venture work. We wish to recognize the assistance gave by our kindred schoolmates to aiding the assets while finishing this undertaking. We truly thank all other people who have assisted us in this venture with working.

Shubham Patil: 22106047

Yuvraj Tiwari: 22106061

Date: